

Documento Técnico: CÉNIT - Sistema de Reservas de Hotel

Administración de Bases de Datos – Proyecto Final (Grupo V)

Desarrollado por:

Josué Abdel Ortiz Deodanes - 00017125
Osmar Alejandro Rodas Rodriguez - 00068125
Kevin Vladimir Rodriguez Alvarado - 00039425
Anderson José Turcios Fores - 00003925
Josué Emiliano Valdés Jacobo - 00003525

1. Descripción del sistema y reglas de negocio

CÉNIT es el sistema con el que un hotel lleva su día a día: qué habitaciones tiene y cuáles están libres, quiénes son sus huéspedes, quién reservó qué, cuánto consumió cada persona durante su estadía y cuánto hay que cobrarle al final.

Este proyecto no arrancó de cero. Partimos de la base de datos que hicimos el ciclo pasado en la materia de Bases de Datos, que funcionaba sobre PostgreSQL. Pasarla a SQL Server fue lo más pesado: había tipos de datos sin equivalente directo, funciones y triggers que reescribir en T-SQL y diferencias en cómo se manejan las secuencias y las restricciones de integridad. Con la base ya migrada, en esta etapa de Administración de Bases de Datos le sumamos seguridad por esquemas y roles, auditoría, índices para las consultas lentas, dimensionamiento, respaldos automáticos y un dashboard en Power BI.

Reglas de negocio

- **Historial de clientes:** un huésped puede reservar tantas veces como quiera, y todas sus visitas quedan registradas.
- **Reservas:** una reservación puede incluir una o varias habitaciones, siempre con fecha de entrada y de salida.
- **Consumos:** mientras dura la estadía (del check-in al check-out), el huésped puede consumir servicios del hotel, como restaurante, lavandería o spa.
- **Facturación:** al hacer el check-out se genera una sola factura que junta la habitación y los servicios, con las temporadas (aumentos) y los descuentos que correspondan.
- **La regla que no se puede romper:** una habitación no puede quedar reservada ni ocupada dos veces en las mismas fechas. De eso se encarga el trigger `trg_verificar_reserva`.

Resumen técnico de la solución

Elemento	Detalle
Motor / base de datos	Microsoft SQL Server · base de datos `HotelDB`
Modelo físico	18 tablas en 3FN repartidas en 4 esquemas de datos (`catalogo`, `operacion`, `facturacion`, `calidad`) y un esquema `reportes` para las vistas analíticas
Lógica de negocio	1 función (`fn_buscar_habitaciones_disponibles`), 1 procedimiento (`sp_calcular_total_factura`) y 4 triggers
Capa analítica	13 vistas en `reportes`, 3 vistas de análisis con funciones ventana y 3 vistas indexadas en `facturacion`
Seguridad	7 roles de base de datos, permisos por esquema, DENY explícito para el auditor y SQL Server Audit sobre facturación
Continuidad	Recovery FULL; respaldos Full semanal, Diferencial diario y Log cada 15 min, automáticos con SQL Server Agent

Nota: en los scripts de seguridad y auditoría la base aparece como `cenitDB`*. Aquí usamos* `HotelDB`*, que es el nombre de los scripts de carga, optimización y respaldos. Es la misma base.*

2. Modelo entidad-relación y esquema relacional

Más adelante van el diagrama entidad-relación en notación Chen (Figura 1) y el esquema relacional en Tercera Forma Normal (Figura 2), que elimina las dependencias parciales y transitivas. En el esquema, (PK) marca las llaves primarias y (FK) las foráneas. Los diagramas van a página completa para que se lean bien, y las relaciones más importantes están resumidas en la sección 2.3.

Versión interactiva de ambos diagramas en draw.io: diagramas_cenit

2.1. Diagrama entidad-relación (notación Chen)

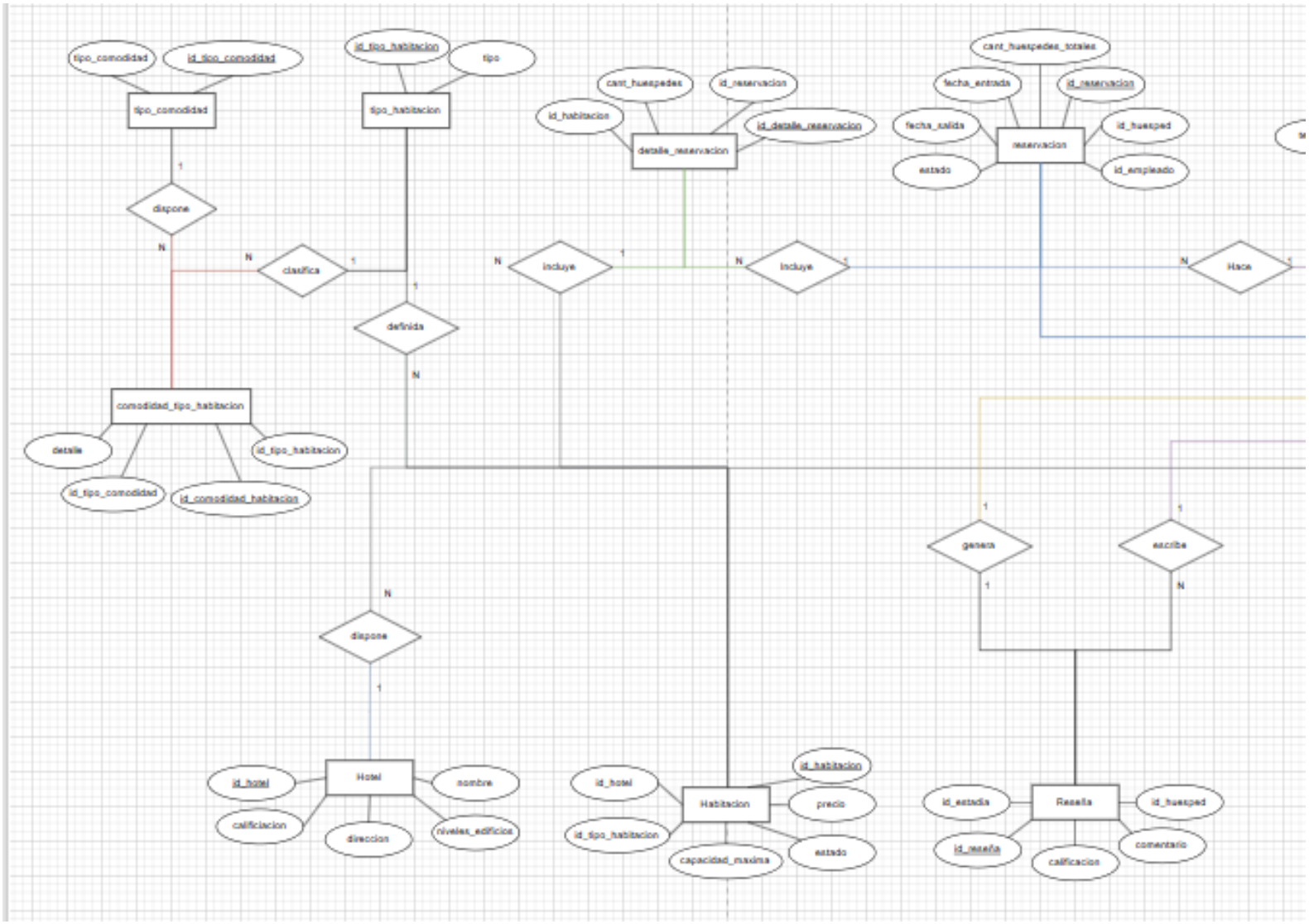


Figura 1a. Diagrama ER en notación Chen (parte 1 de 2).

2.1. Diagrama entidad-relación (continuación)

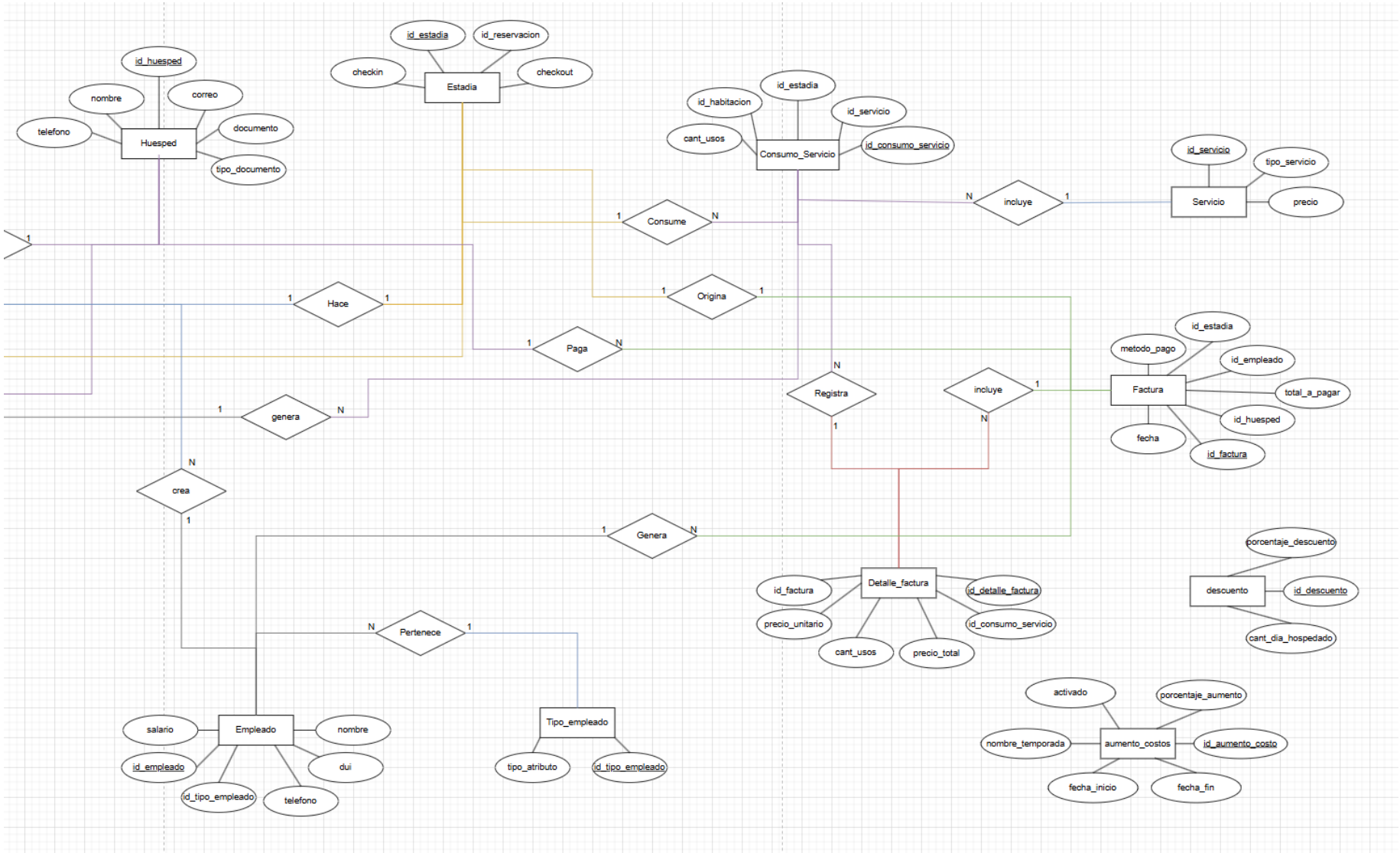


Figura 1b. Diagrama ER en notación Chen (parte 2 de 2).

2.2. Esquema relacional normalizado (3FN)

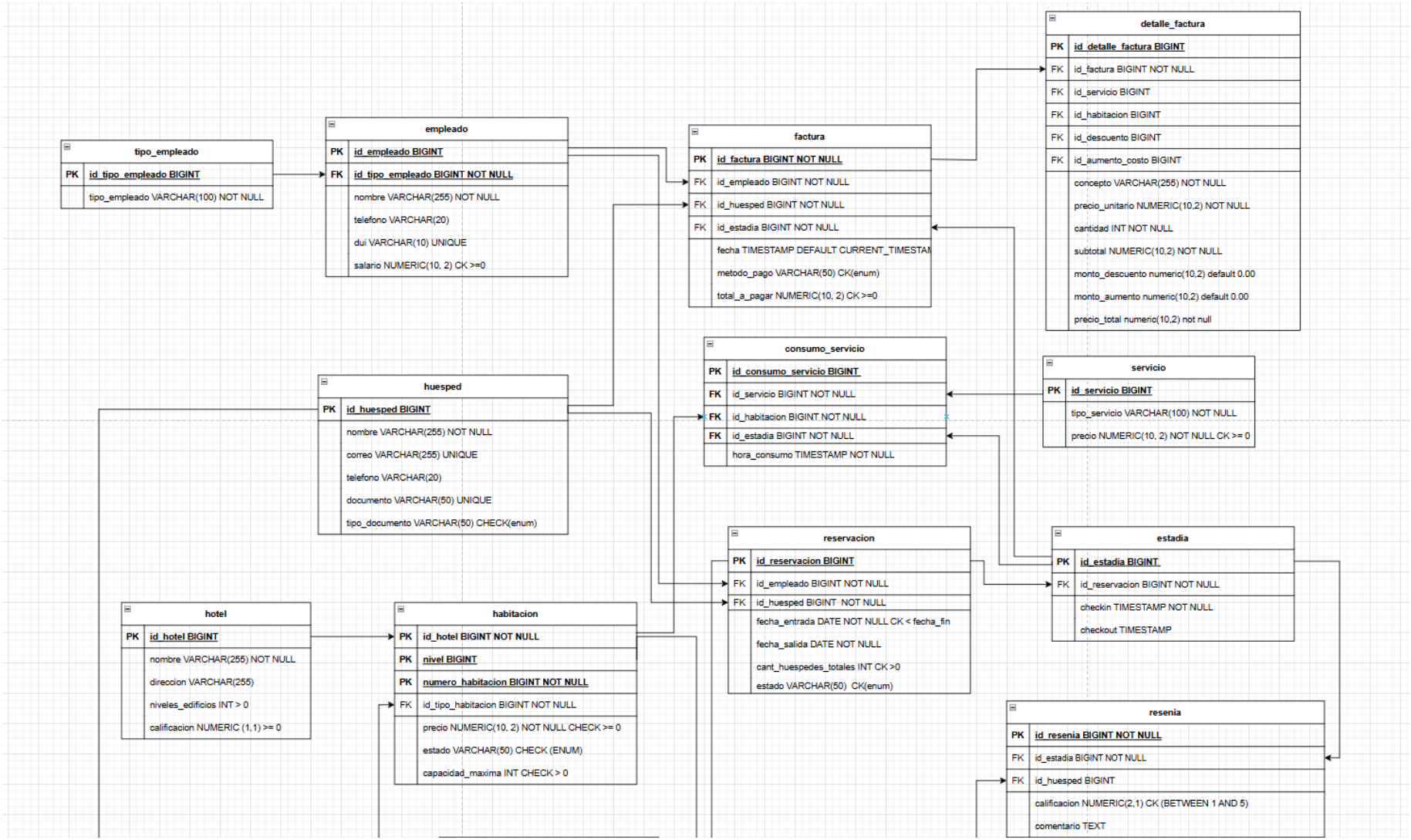


Figura 2a. Esquema relacional en 3FN (parte 1 de 2).

2.2. Esquema relacional normalizado (continuación)

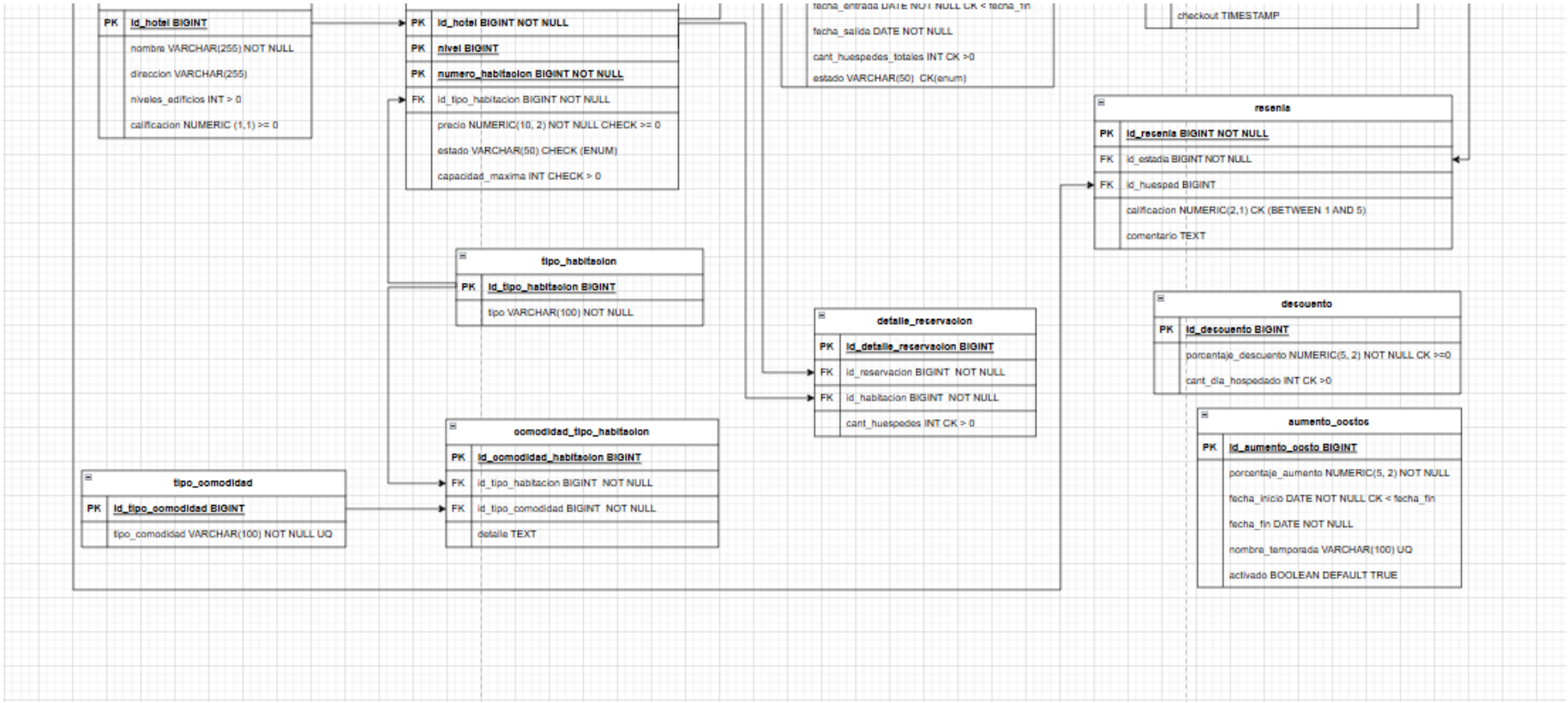


Figura 2b. Esquema relacional en 3FN (parte 2 de 2).

2.3. Relaciones principales del modelo

Relación	Cardinalidad	Implementación
hotel – habitacion	1 : N	`habitacion.id_hotel`; nivel limitado por `niveles_edificios` (trigger)
tipo_habitacion – habitacion	1 : N	`habitacion.id_tipo_habitacion`
tipo_habitacion – tipo_comodidad	N : M	Tabla intermedia `comodidad_tipo_habitacion`
huesped – reservacion	1 : N	`reservacion.id_huesped`; la gestiona un `empleado`
reservacion – habitacion	N : M	Tabla intermedia `detalle_reservacion` (fechas de entrada y salida)
reservacion – estadia	1 : N	`estadia.id_reservacion` (check-in / check-out)
estadia – servicio	N : M	Tabla `consumo_servicio` (servicio, habitación y estadía)
estadia – factura	1 : N	`factura` única por (empleado, huésped, estadía); trigger `trg_checkout_factura`
factura – detalle_factura	1 : N	Cada línea cobra un servicio o una habitación (`ck_origen_cobro`)
estadia / huesped – resenia	1 : N	Una reseña por (estadía, huésped); recalcula `hotel.calificacion`

3. Diccionario de datos

Las 18 tablas del modelo en SQL Server están repartidas en cuatro esquemas, según lo que hace cada una. Así podemos dar permisos por esquema y no tabla por tabla. En cada tabla, la columna «Llave / Nulo» dice si el campo es llave primaria (PK) o foránea (FK, con la tabla a la que apunta) y si acepta valores nulos.

Esquema	Propósito	Tablas
catalogo	Datos de referencia y configuración, de baja frecuencia de cambio.	hotel, tipo_habitacion, tipo_comodidad, tipo_empleado, servicio, descuento, aumento_costos, comodidad_tipo_habitacion
operacion	El día a día del hotel: habitaciones, personal, huéspedes y todo el ciclo de reservación y estadía.	habitacion, empleado, huesped, reservacion, detalle_reservacion, estadia, consumo_servicio
facturacion	Emisión y detalle de las facturas.	factura, detalle_factura
calidad	Opiniones de los huéspedes sobre su estadía.	resenia

3.1. Tabla: catalogo.hotel

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
`id_hotel`	bigint	PK / No Nulo	Identity	Identificador único del hotel.
`nombre`	nvarchar(255)	No Nulo	Unique	Nombre comercial del hotel.
`direccion`	nvarchar(255)	Nulo	-	Ubicación física del hotel.
`niveles_edificios`	int	No Nulo	Check (> 0)	Cantidad de niveles del edificio; limita el nivel de sus habitaciones (trigger tg_check_nivel_habitacion).
`calificacion`	decimal(2,1)	Nulo	Check (entre 1 y 5)	Promedio de las reseñas del hotel; se recalcula con el trigger trg_actualizar_calificacion.
`descripcion`	nvarchar(max)	Nulo	-	Descripción general del hotel.

3.2. Tabla: catalogo.tipo_habitacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
`id_tipo_habitacion`	bigint	PK / No Nulo	Identity	Identificador único del tipo de habitación.
`tipo_habitacion`	nvarchar(100)	No Nulo	Unique	Nombre del tipo de habitación (p. ej., Suite Ejecutiva).

3.3. Tabla: catalogo.tipo_comodidad

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
`id_tipo_comodidad`	bigint	PK / No Nulo	Identity	Identificador único de la comodidad.
`tipo_comodidad`	nvarchar(100)	No Nulo	Unique	Nombre de la comodidad (p. ej., Jacuzzi Privado).

3.4. Tabla: catalogo.tipo_empleado

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_tipo_empleado	bigint	PK / No Nulo	Identity	Identificador único del tipo de empleado.
tipo_empleado	nvarchar(100)	No Nulo	Unique	Rol o cargo del empleado (p. ej., Recepcionista, Gerente).

3.5. Tabla: catalogo.servicio

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_servicio	bigint	PK / No Nulo	Identity	Identificador único del servicio.
tipo_servicio	nvarchar(100)	No Nulo	Unique	Nombre del servicio adicional (p. ej., Lavandería).
precio	decimal(10,2)	No Nulo	Check (>= 0)	Precio unitario del servicio.

3.6. Tabla: catalogo.descuento

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_descuento	bigint	PK / No Nulo	Identity	Identificador único del descuento.
porcentaje_descuento	decimal(5,2)	No Nulo	Check (>= 0 AND <= 100)	Porcentaje de descuento aplicable a la estadía.
cant_dia_hospedado	int	Nulo	Check (> 0)	Noches mínimas para que aplique el descuento; al facturar se usa el umbral más alto que no supere la estadía.

3.7. Tabla: catalogo.aumento_costos

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_aumento_costo	bigint	PK / No Nulo	Identity	Identificador único del aumento por temporada.
porcentaje_aumento	decimal(5,2)	No Nulo	Check (>= 0)	Porcentaje a incrementar sobre la tarifa base.
fecha_inicio	date	No Nulo	Check (fecha_inicio < fecha_fin)	Fecha de inicio de la temporada (inclusive).
fecha_fin	date	No Nulo	-	Fecha de finalización de la temporada (inclusive).
nombre_temporada	nvarchar(100)	Nulo	Unique	Nombre descriptivo de la temporada (p. ej., Semana Santa 2026).
activado	bit	No Nulo	Default (1)	Solo las temporadas activas se consideran al calcular la factura.

3.8. Tabla: catalogo.comodidad_tipo_habitacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_comodidad_habitacion	bigint	PK / No Nulo	Identity	Identificador único de la asociación.
id_tipo_habitacion	bigint	FK / No Nulo → catalogo.tipo_habitacion	-	Tipo de habitación al que pertenece la comodidad.
id_tipo_comodidad	bigint	FK / No Nulo → catalogo.tipo_comodidad	-	Comodidad que ofrece el tipo de habitación.
detalle	nvarchar(max)	No Nulo	-	Condición de la comodidad (p. ej., Incluido en la tarifa).

Nota: une comodidades con tipos de habitación, no con habitaciones individuales; por eso no apunta a operacion.habitacion.

3.9. Tabla: operacion.habitacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_habitacion	bigint	PK / No Nulo	Identity	Identificador único de la habitación.
id_hotel	bigint	FK / No Nulo → catalogo.hotel	-	Hotel al que pertenece la habitación.
nivel	int	No Nulo	Check (>= 0)	Nivel del edificio donde se ubica; no puede superar los niveles del hotel.
numero_habitacion	int	No Nulo	Check (>= 0)	Número de la habitación dentro del nivel.
id_tipo_habitacion	bigint	FK / No Nulo → catalogo.tipo_habitacion	-	Clasificación de la habitación.

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
precio	decimal(10,2)	No Nulo	Check (>= 0)	Tarifa por noche.
estado	nvarchar(50)	No Nulo	Check (DISPONIBLE, OCUPADA, MANTENIMIENTO)	Estado operativo de la habitación.
capacidad_maxima	int	No Nulo	Check (entre 1 y 15)	Cantidad máxima de huéspedes.
descripcion	nvarchar(max)	Nulo	-	Descripción de la habitación.

Nota: la combinación (id_hotel, nivel, numero_habitacion) es única.

3.10. Tabla: operacion.empleado

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_empleado	bigint	PK / No Nulo	Identity	Identificador único del empleado.
id_tipo_empleado	bigint	FK / No Nulo → catalogo.tipo_empleado	-	Cargo del empleado.
nombre	nvarchar(255)	No Nulo	-	Nombre completo del empleado.
correo	nvarchar(100)	No Nulo	Unique, Check (formato de correo)	Correo electrónico institucional.
telefono	nvarchar(20)	Nulo	Check (+503 seguido de 2, 6 o 7 y 7 dígitos)	Teléfono de contacto salvadoreño.
dui	nvarchar(10)	No Nulo	Unique, Check (8 dígitos, guion, 1 dígito)	Documento Único de Identidad.
salario	decimal(10,2)	No Nulo	Check (>= 0)	Salario mensual del empleado.

3.11. Tabla: operacion.huesped

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_huesped	bigint	PK / No Nulo	Identity	Identificador único del huésped.
nombre	nvarchar(255)	No Nulo	-	Nombre completo del huésped.
correo	nvarchar(255)	No Nulo	Unique, Check (formato de correo)	Correo electrónico de contacto.
telefono	nvarchar(20)	No Nulo	Check (7 a 20 caracteres: dígitos, espacios, guion o +)	Teléfono de contacto.
documento	nvarchar(50)	No Nulo	Unique	Número del documento de identidad; su formato depende del tipo de documento.
tipo_documento	nvarchar(50)	No Nulo	Check (DUI, PASAPORTE, CONSTANCIA DE RESIDENCIA)	Tipo de documento presentado.

3.12. Tabla: operacion.reservacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_reservacion	bigint	PK / No Nulo	Identity	Identificador único de la reservación.
id_empleado	bigint	FK / No Nulo → operacion.empleado	-	Empleado que gestionó la reservación.
id_huesped	bigint	FK / No Nulo → operacion.huesped	-	Huésped que solicita la reservación.
cant_huespedes_totales	int	No Nulo	Check (> 0)	Total de huéspedes de la reservación.
estado	nvarchar(50)	No Nulo	Check (PENDIENTE, CONFIRMADA, CANCELADA, RECHAZADA, COMPLETADA)	Estado actual de la reservación.

3.13. Tabla: operacion.detalle_reservacion

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_detalle_reservacion	bigint	PK / No Nulo	Identity	Identificador único del detalle.
id_reservacion	bigint	FK / No Nulo → operacion.reservacion	-	Reservación a la que pertenece.
id_habitacion	bigint	FK / No Nulo → operacion.habitacion	-	Habitación reservada.
cant_huespedes	int	No Nulo	Check (> 0)	Huéspedes alojados en esta habitación.

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
fecha_entrada	date	No Nulo	-	Fecha de entrada.
fecha_salida	date	No Nulo	Check (fecha_entrada < fecha_salida)	Fecha de salida.

Nota: el trigger `trg_verificar_reserva` rechaza el registro si la habitación ya tiene otra reserva activa (que no esté cancelada ni rechazada) en fechas que se cruzan.

3.14. Tabla: operacion.estadia

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_estadia	bigint	PK / No Nulo	Identity	Identificador único de la estadía.
id_reservacion	bigint	FK / No Nulo → operacion.reservacion	-	Reservación que origina la estadía.
checkin	datetime2(3)	No Nulo	Default (SYSDATETIME())	Fecha y hora de ingreso.
checkout	datetime2(3)	Nulo	Check (nulo o checkin < checkout)	Fecha y hora de salida; al registrarla, el trigger trg_checkout_factura genera la factura.

3.15. Tabla: operacion.consumo_servicio

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_consumo_servicio	bigint	PK / No Nulo	Identity	Identificador único del consumo.
id_servicio	bigint	FK / No Nulo → catalogo.servicio	-	Servicio consumido.
id_habitacion	bigint	FK / No Nulo → operacion.habitacion	-	Habitación desde la que se consumió.
id_estadia	bigint	FK / No Nulo → operacion.estadia	-	Estadía a la que se carga el consumo.
hora_consumo	datetime2(3)	No Nulo	Default (SYSDATETIME())	Fecha y hora del consumo.

3.16. Tabla: facturacion.factura

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_factura	bigint	PK / No Nulo	Identity	Identificador único de la factura.
id_empleado	bigint	FK / No Nulo → operacion.empleado	-	Empleado que emite la factura.
id_huesped	bigint	FK / No Nulo → operacion.huesped	-	Huésped facturado.
id_estadia	bigint	FK / No Nulo → operacion.estadia	-	Estadía que se factura.
fecha	datetime2(3)	No Nulo	Default (SYSDATETIME())	Fecha y hora de emisión.
metodo_pago	nvarchar(50)	No Nulo	Check (EFECTIVO, TRANSFERENCIA, TARJETA, BITCOIN, PAYPAL)	Forma de pago utilizada.
total_a_pagar	decimal(10,2)	No Nulo	Check (>= 0)	Suma de precio_total de su detalle; la calcula sp_calcular_total_factura.

Nota: la combinación (id_empleado, id_huesped, id_estadia) es única.

3.17. Tabla: facturacion.detalle_factura

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_detalle_factura	bigint	PK / No Nulo	Identity	Identificador único de la línea de factura.
id_factura	bigint	FK / No Nulo → facturacion.factura	-	Factura a la que pertenece la línea.
id_servicio	bigint	FK / Nulo → catalogo.servicio	-	Servicio cobrado (línea de servicio).
id_habitacion	bigint	FK / Nulo → operacion.habitacion	-	Habitación cobrada (línea de hospedaje).
id_descuento	bigint	FK / Nulo → catalogo.descuento	-	Descuento aplicado, si corresponde.
id_aumento_costo	bigint	FK / Nulo → catalogo.aumento_costos	-	Temporada aplicada, si corresponde.

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
concepto	nvarchar(255)	No Nulo	-	Descripción de la línea.
precio_unitario	decimal(10,2)	No Nulo	Check (>= 0)	Precio por noche o por unidad.
cantidad	int	No Nulo	Check (>= 1)	Noches o unidades cobradas.
subtotal	decimal(10,2)	No Nulo	Check (>= 0)	Calculado: precio_unitario × cantidad.
monto_descuento	decimal(10,2)	No Nulo	Default (0), Check (>= 0)	Monto descontado.
monto_aumento	decimal(10,2)	No Nulo	Default (0), Check (>= 0)	Monto añadido por temporada.
precio_total	decimal(10,2)	No Nulo	Check (>= 0)	Calculado: (subtotal + monto_aumento) - monto_descuento.

Nota: la restricción `ck_origen_cobro` exige que justo uno entre id_servicio e id_habitacion tenga valor. Cada línea cobra un servicio o una noche de hospedaje, nunca las dos cosas ni ninguna.

3.18. Tabla: calidad.resenia

Columna	Tipo	Llave / Nulo	Restricciones	Descripción
id_resenia	bigint	PK / No Nulo	Identity	Identificador único de la reseña.
id_estadia	bigint	FK / No Nulo → operacion.estadia	-	Estadía reseñada.
id_huesped	bigint	FK / No Nulo → operacion.huesped	-	Huésped que escribe la reseña.
calificacion	decimal(2,1)	Nulo	Check (entre 1 y 5)	Calificación otorgada.
comentario	nvarchar(max)	Nulo	-	Comentario del huésped.

Nota: la combinación (id_estadia, id_huesped) es única. Además, el trigger `trg_actualizar_calificacion` no deja registrar una segunda reseña para la misma estadía y recalcula la calificación del hotel.

3.19. Notas de diseño

- **Llaves primarias:** todas son `bigint` con Identity y nadie las modifica a mano.
- **Llaves foráneas:** todas usan `ON UPDATE NO ACTION` y `ON DELETE NO ACTION`. SQL Server no deja usar `ON UPDATE CASCADE` en este modelo porque habría varias rutas de cascada (por ejemplo, `huesped` llega a la factura directo y también pasando por `reservacion` y `estadia`). Como las llaves nunca se modifican, en la práctica no cambia nada.
- **Esquemas:** separar todo en `catalogo`, `operacion`, `facturacion`, `calidad` y `reportes` nos deja dar permisos por esquema a cada rol.
- **Lógica de negocio:** una función (`operacion.fn_buscar_habitaciones_disponibles`), un procedimiento (`facturacion.sp_calcular_total_factura`) y cuatro triggers (`tg_check_nivel_habitacion`, `trg_verificar_reserva`, `trg_checkout_factura` y `trg_actualizar_calificacion`).

4. Políticas de seguridad implementadas

Para la seguridad seguimos una idea simple: que cada persona vea y toque solo lo que necesita para hacer su trabajo (principio de menor privilegio). Lo armamos con cuatro piezas: los objetos separados en esquemas, roles de base de datos con permisos por esquema, un DENY explícito para el rol de auditoría y SQL Server Audit vigilando las tablas de facturación.

4.1. Esquemas y justificación

Esquema	Contenido	Justificación
catalogo	hotel, servicio, tipo_habitacion, tipo_comodidad, tipo_empleado, descuento, aumento_costos, comodidad_tipo_habitacion	Reúne las reglas de negocio, los catálogos y las tarifas. Casi todos los roles solo leen (SELECT); escribir queda para gerencia.
operacion	huesped, empleado, habitacion, reservacion, detalle_reservacion, estadia, consumo_servicio	Es el corazón transaccional del hotel: ahí se concentran la mayoría de las lecturas, inserciones y actualizaciones.
facturacion	factura, detalle_factura	Datos financieros delicados y el rastro de cada cobro. Solo caja y gerencia tienen acceso.

Esquema	Contenido	Justificación
calidad	resenia	Deja las opiniones de los clientes fuera del núcleo transaccional, para que analizar la satisfacción no le quite recursos a la operación cuando el hotel está lleno.
reportes	13 vistas analíticas (v_ingresos_mes, v_tasa_ocupacion_mensual, v_resumen_hotel, v_factura_completa, v_gasto_historica, v_habitaciones_disponibles, etc.)	Es la capa para análisis: se puede consultar sin dar acceso directo a las tablas transaccionales.

```
CREATE SCHEMA catalogo;     CREATE SCHEMA operacion;
CREATE SCHEMA facturacion;  CREATE SCHEMA calidad;   CREATE SCHEMA reportes;

-- Traslado de objetos desde dbo (ejemplo; se aplicó a las 18 tablas y 13 vistas)
ALTER SCHEMA facturacion TRANSFER dbo.factura;
ALTER SCHEMA reportes    TRANSFER dbo.v_ingresos_mes;

-- Verificación de la asignación de esquemas
SELECT s.name AS Esquema, t.name AS NombreTabla
FROM sys.tables t JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE s.name IN ('calidad','catalogo','operacion','facturacion','reportes')
ORDER BY s.name, t.name;
```

4.2. Roles, usuarios y permisos

Definimos siete roles de base de datos. Cada uno tiene su login de servidor y su usuario de base de datos. Las contraseñas no las ponemos en el documento.

Rol	Usuario / login	Responsabilidad y permisos
rol_programador (Programador IT)	usr_Rodas_Rodriguez / login_dev	Desarrolla y optimiza. Puede crear y modificar objetos (ALTER, CREATE TABLE/VIEW/PROCEDURE/FUNCTION) en todos los esquemas y hace DML en catalogo, operacion, calidad y reportes; en facturacion solo lee. También puede usar EXECUTE, VIEW DEFINITION, SHOWPLAN, BACKUP DATABASE y BACKUP LOG.
rol_gerente (Gerente)	usr_Alexander_Vazquez / login_gerente	Supervisa todo y maneja las tarifas. SELECT, INSERT, UPDATE y DELETE en catalogo, operacion, facturacion y calidad; solo SELECT en reportes; puede ejecutar rutinas. No puede crear ni modificar estructuras (sin DDL).
rol_recepcionista (Recepcionista)	usr_Kenia_Abigail / login_recepcion	Atiende el mostrador. SELECT en catalogo; SELECT, INSERT y UPDATE en huesped, reservacion, detalle_reservacion y estadia; SELECT en habitacion y consumo_servicio; SELECT e INSERT en facturacion y calidad; y las vistas operativas de reportes.
rol_cajero (Cajero)	usr_Cesar_Ramirez / login_cajero	Lleva la caja y los cobros. SELECT en servicio, descuento, aumento_costos, huesped, estadia y consumo_servicio; SELECT, INSERT y UPDATE en facturacion; vistas financieras. No tiene acceso a calidad.
rol_auditor (Auditor)	usr_Oscar_Olivo / login_auditor	Es el control interno. SELECT en todos los esquemas y VIEW DEFINITION, con DENY de INSERT, UPDATE y DELETE para que no pueda cambiar nada.
rol_mesero_servicio (Mesero / Servicios)	usr_Bryan_Ortiz / login_mesero (y usr_Sofia_Guardado, supervisora)	Trabaja en los puntos de venta. SELECT en servicio, habitacion y estadia; SELECT e INSERT en consumo_servicio. No ve facturacion, calidad ni reportes.
rol_atencion_cliente (Atención al Cliente)	usr_Steven_Roque / login_atencion	Cuida la satisfacción después de la estadía. SELECT en hotel, tipo_habitacion, huesped y estadia; SELECT, INSERT y UPDATE en calidad.resenia (sin DELETE, para no perder evidencia); y las vistas cualitativas.

Matriz resumen de permisos por esquema

Rol	catalogo	operacion	facturacion	calidad	reportes
rol_programador	ALL (DDL/DML)	ALL (DDL/DML)	DDL / SELECT	ALL (DDL/DML)	ALL (DDL/DML)
rol_gerente	DML completo	DML completo	DML completo	DML completo	SELECT
rol_recepcionista	SELECT	SELECT, INSERT, UPDATE	SELECT, INSERT	SELECT, INSERT	SELECT (operativo)

Rol	catalogo	operacion	facturacion	calidad	reportes
rol_cajero	SELECT	SELECT	SELECT, INSERT, UPDATE	Sin acceso	SELECT (financiero)
rol_auditor	SELECT (DENY DML)	SELECT (DENY DML)	SELECT (DENY DML)	SELECT (DENY DML)	SELECT (DENY DML)
rol_mesero_servicio	SELECT (servicio)	SELECT, INSERT (consumos)	Sin acceso	Sin acceso	Sin acceso
rol_atencion_cliente	SELECT (general)	SELECT (huesped/estadia)	Sin acceso	SELECT, INSERT, UPDATE	SELECT (cualitativo)

4.3. Auditoría con SQL Server Audit

Para poder rastrear qué pasa con los datos financieros configuramos una auditoría a nivel de servidor (`Audit_Cenit_Servidor`). Guarda en archivo (`/audit/Audit_Cenit`, hasta 100 MB por archivo y 5 archivos de rotación, con `ON_FAILURE = CONTINUE`). Encima va una especificación de auditoría de base de datos (`AuditSpec_Facturacion_Sensible`) que anota todos los **INSERT y UPDATE** que cualquier usuario (`public`) haga sobre `facturacion.factura` y `facturacion.detalle_factura`. Los eventos se leen con `sys.fn_get_audit_file`, que devuelve fecha (UTC), login, usuario, acción, objeto y la sentencia ejecutada (Figura 9).

```
CREATE SERVER AUDIT Audit_Cenit_Servidor
  TO FILE (FILEPATH = '/audit/Audit_Cenit', MAXSIZE = 100 MB, MAX_ROLLOVER_FILES = 5)
  WITH (QUEUE_DELAY = 1000, ON_FAILURE = CONTINUE);
ALTER SERVER AUDIT Audit_Cenit_Servidor WITH (STATE = ON);

CREATE DATABASE AUDIT SPECIFICATION AuditSpec_Facturacion_Sensible
  FOR SERVER AUDIT Audit_Cenit_Servidor
  ADD (INSERT ON facturacion.factura BY public),
  ADD (UPDATE ON facturacion.factura BY public),
  ADD (INSERT ON facturacion.detalle_factura BY public),
  ADD (UPDATE ON facturacion.detalle_factura BY public)
  WITH (STATE = ON);
```

4.4. Evidencia de pruebas de roles y permisos

Probamos cada rol con `EXECUTE AS USER`: una operación que debía funcionar ([ÉXITO]) y otra que debía ser rechazada ([FALLO ESPERADO]). Todo respondió como dice la matriz de permisos.

Prueba	Usuario (rol)	Operación permitida	Operación denegada
1	usr_Rodas_Rodriguez (programador)	CREATE/DROP TABLE en catalogo; SELECT en huesped	DELETE en huesped → Msg 229
2	usr_Alexander_Vazquez (gerente)	SELECT en v_factura_completa y resenia	CREATE TABLE → Msg 262
3	usr_Kenia_Abigail (recepcionista)	SELECT en v_habitaciones_disponibles y huesped	UPDATE habitacion y UPDATE factura → Msg 229
4	usr_Cesar_Ramirez (cajero)	UPDATE de metodo_pago en factura (1 fila)	SELECT en calidad.resenia → Msg 229
5	usr_Oscar_Olivo (auditor)	SELECT en hotel, factura y v_ingresos_mes	INSERT en resenia y DELETE en servicio → Msg 229

```sql
-- ============================================================================
-- PRUEBA 1: Prueba de Éxito: Gestión estructural (DDL) y lectura de datos en esquemas.
-- Prueba de Error: Intento de modificación/borrado de datos (DML) o elevación a nivel de servidor.
-- ============================================================================
EXECUTE AS USER = 'usr_Rodas_Rodriguez';

-- [ÉXITO] Crear y consultar un objeto DDL estructural
CREATE TABLE catalogo.prueba_desarrollo (
    id INT IDENTITY PRIMARY KEY,
    nombre VARCHAR(50)
);
SELECT TOP 2 * FROM operacion.huesped;

-- Limpieza
DROP TABLE catalogo.prueba_desarrollo;

-- [FALLO ESPERADO 2]: Intentar eliminar una factura
-- Error: Msg 229, The DELETE permission was denied on the object 'factura'
DELETE FROM facturacion.factura
WHERE id_factura = 1;
GO
```

```
Mensajes
Mens. 229, Nivel 14, Estado 5, Línea 113
The DELETE permission was denied on the object 'factura', database 'HotelDB', schema 'facturacion'.

Hora de finalización: 2026-11-09T23:31:36.9078795-06:00
```

Figura 3. Prueba 1 – Programador IT: DDL permitido y DELETE de datos denegado.

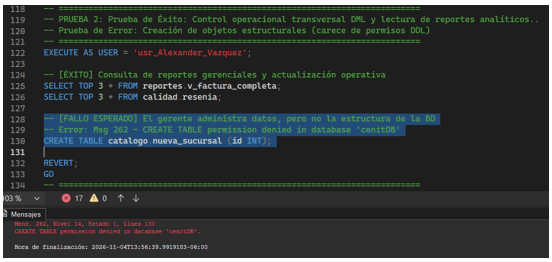


Figura 4. Prueba 2 – Gerente: administra datos, pero CREATE TABLE es denegado (sin DDL).

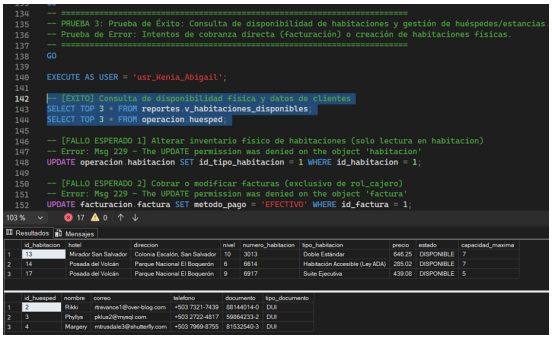


Figura 5. Prueba 3 – Recepcionista: consulta disponibilidad y huéspedes; no puede alterar habitaciones ni facturas.

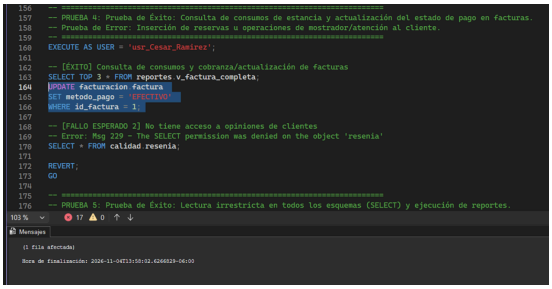


Figura 6. Prueba 4 – Cajero: actualiza el método de pago de una factura (1 fila afectada).

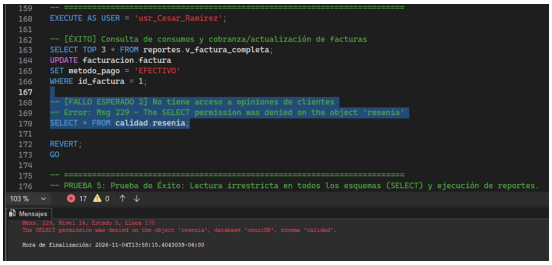


Figura 7. Prueba 4 (cont.) – Cajero: acceso denegado a calidad.resenia.

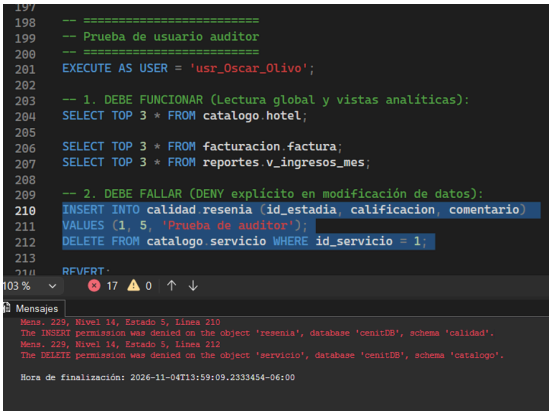


Figura 8. Prueba 5 – Auditor: lectura global permitida; INSERT y DELETE denegados por DENY explícito.

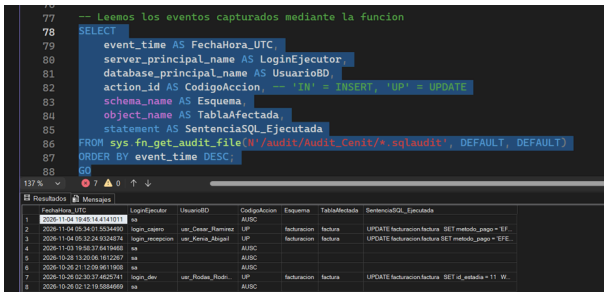


Figura 9. Registro de SQL Server Audit: UPDATE sobre facturacion.factura por login_cajero, login_recepcion y login_dev.

Observaciones y mejoras recomendadas

- **Contraseñas:** en el entorno de la clase creamos los logins con `CHECK_POLICY = OFF`. En producción hay que activar `CHECK_POLICY` y `CHECK_EXPIRATION` y rotar las credenciales.
- **TRUSTWORTHY:** el script de auditoría activa `TRUSTWORTHY ON` para ejecutar módulos con `EXECUTE AS`. En producción sería mejor firmar los módulos con un certificado y dejar esa opción apagada.
- **Protección de datos:** más adelante convendría cifrar los respaldos y activar TDE en la base.

5. Optimización de consultas e índices

5.1. Metodología

Optimizamos tres consultas analíticas de facturación, armadas como vistas con **funciones ventana** (`DENSE_RANK`, `RANK`, `LAG`, `ROW_NUMBER`, `SUM() OVER`). El problema es que la base real tiene apenas ~190 facturas y ~1,400 líneas de detalle: con tan pocos datos un escaneo completo es lo más barato y ningún índice se lucería. Por eso generamos una **carga sintética de ~200,000 facturas** (de 1 a 4 líneas cada una, 75 % de habitación y 25 % de servicio) con el script `00 carga sintetica hoteldb.sql`, que además permite devolver los datos a como estaban.

Seguimos cuatro pasos: (1) crear las vistas base, (2) medirlas **sin índices** con `SET STATISTICS IO ON` y `SET STATISTICS TIME ON`, (3) crear los índices y las vistas indexadas, y (4) repetir las mismas consultas para comparar. Nos fijamos sobre todo en las **lecturas lógicas** (páginas de 8 KB leídas), porque casi no cambian de una ejecución a otra. El tiempo de CPU lo usamos como apoyo.

5.2. Índices y objetos aplicados

Objeto	Tipo	Definición	Propósito
IX_factura_huesped	No agrupado	factura (id_huesped) INCLUDE (total_a_pagar)	Agrupar el gasto por huésped sin tocar la tabla base
IX_factura_fecha	No agrupado	factura (fecha) INCLUDE (total_a_pagar)	Index seek por rango de fechas (consulta 2)
IX_detfac_habitacion	No agrupado filtrado	detalle_factura (id_habitacion) INCLUDE (precio_total) WHERE id_habitacion IS NOT NULL	Ingreso por habitación; excluye las líneas de servicio
IX_detfac_factura	No agrupado filtrado	detalle_factura (id_factura) INCLUDE (id_habitacion, precio_total) WHERE id_habitacion IS NOT NULL	Traer solo las líneas de las facturas del rango
ivw_gasto_huesped	Vista indexada	UNIQUE CLUSTERED (id_huesped)	Agregado precalculado: 1 fila por huésped (consulta 1)
ivw_monto_hotel_factura	Vista indexada	UNIQUE CLUSTERED (id_factura, id_hotel)	Monto por factura y hotel precalculado (soporte consulta 2)
ivw_ingreso_habitacion	Vista indexada	UNIQUE CLUSTERED (id_habitacion)	1 fila por habitación, sin importar las líneas facturadas (consulta 3)
IX_ivw_ingreso_habitacion_top	No agrupado sobre vista	(id_hotel, ingresos DESC, id_habitacion) INCLUDE (...)	Mismo orden del ROW_NUMBER: el top 3 por hotel lee pocas filas

```
CREATE VIEW facturacion.ivw_gasto_huesped WITH SCHEMABINDING AS
SELECT f.id_huesped, SUM(f.total_a_pagar) AS gasto_total, COUNT_BIG(*) AS num_facturas
FROM facturacion.factura AS f GROUP BY f.id_huesped;
GO
CREATE UNIQUE CLUSTERED INDEX CIX_ivw_gasto_huesped
  ON facturacion.ivw_gasto_huesped (id_huesped);
```

Las vistas indexadas piden `WITH SCHEMABINDING` y `COUNT_BIG(*)`. En las ediciones Enterprise y Developer el optimizador las usa solo; en las demás hay que llamarlas con el hint `WITH (NOEXPAND)`.

5.3. Consulta 1 – Ranking de gasto por huésped

Vista: `facturacion.vw_ranking_gastos_huespedes`. Calcula cuánto gastó cada huésped y arma el ranking con `DENSE_RANK()` global, `RANK()` por tipo de documento y el porcentaje sobre el total con `SUM() OVER ()`.

Problema (sin índice): cada vez que se ejecuta suma todas las facturas, o sea que escanea `factura` completa (1,938 páginas).

Solución aplicada: una vista indexada, `ivw_gasto_huesped` (una fila por huésped), apoyada en `IX_factura_huesped`. El optimizador lee la vista y deja tranquila la tabla.

Métrica	Sin índice	Con índice	Mejora
Lecturas lógicas de la tabla principal	factura: 1,938	ivw_gasto_huesped: 6	99.7 %
Lecturas lógicas totales (incl. huesped: 26)	1,964	32	98.4 %
CPU en ejecución	21 ms	5 ms	76 %
Tiempo transcurrido	24 ms	5 ms	79 %

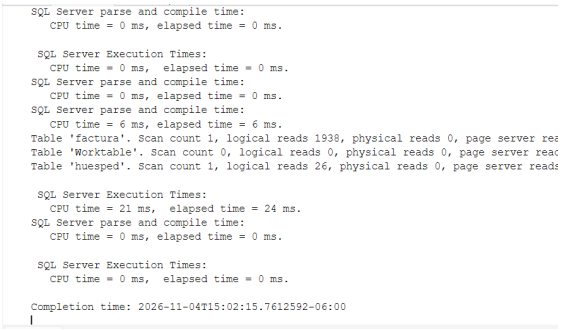


Figura 10a. Consulta 1 sin índice (STATISTICS IO/TIME).

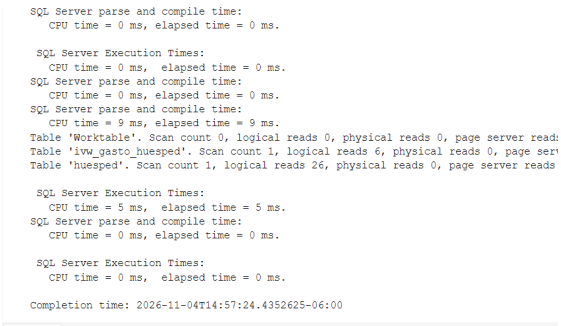


Figura 10b. Consulta 1 con vista indexada.

5.4. Consulta 2 – Variación mensual de ingresos por hotel

Vista: `facturacion.vw_variacion_mensual_ingresos_hotel`. Reparte el total de cada factura entre los hoteles según sus líneas de habitación y compara cada mes con el anterior usando `LAG()`.

Problema (sin índice): el filtro por mes se aplica después de la ventana, así que aunque se pidan solo 3 meses la vista lee todas las facturas y todas las líneas (`detalle_factura`: 4,736 páginas; `factura`: 2,038).

Solución aplicada: una función con valores de tabla, `fn_variacion_mensual_ingresos_hotel(@desde, @hasta)`, que filtra las facturas antes de la ventana (incluye un mes anterior para el `LAG`) y se apoya en `IX_factura_fecha` (seek) e `IX_detfac_factura`. Con `EXCEPT` comprobamos que devuelve lo mismo que la vista.

Métrica	Sin índice	Con índice	Mejora
Lecturas lógicas totales	6,794	618	90.9 %
detalle_factura	4,736 (13 escaneos)	324 (94 búsquedas)	93.2 %
factura	2,038	8	99.6 %
CPU en ejecución	620 ms	4 ms	99.4 %
Tiempo transcurrido	105 ms	4 ms	96.2 %

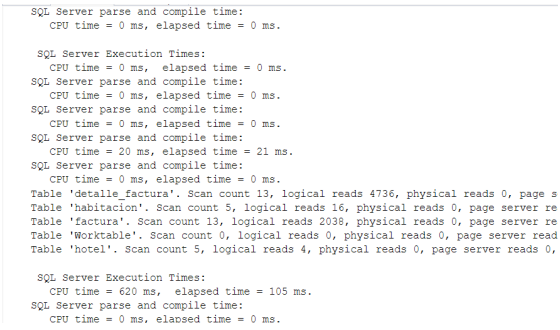


Figura 11a. Consulta 2 sin índice (rango 2026-03 a 2026-06).

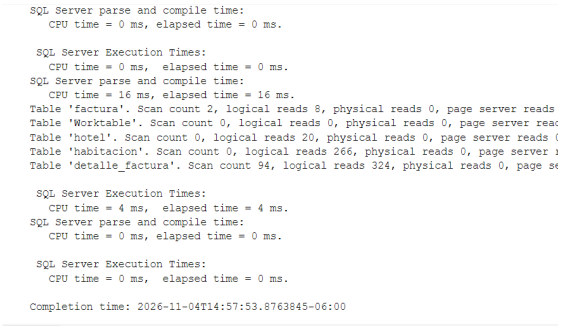


Figura 11b. Consulta 2 con función filtrada e índices.

5.5. Consulta 3 – Top 3 habitaciones más rentables por hotel

Vista: `facturacion.vw_top3_habitaciones_rentables`. Suma los ingresos de cada habitación y se queda con las tres mejores de cada hotel mediante `ROW_NUMBER() OVER (PARTITION BY id_hotel ORDER BY ingresos DESC)`.

Problema (sin índice): lee todas las líneas de habitación de `detalle_factura` (4,504 páginas) para devolver unas ~9 filas.

Solución aplicada: la vista indexada `ivw_ingreso_habitacion` (una fila por habitación) con el índice `IX_ivw_ingreso_habitacion_top`, que está en el mismo orden que la ventana.

Métrica	Sin índice	Con índice	Mejora
Lecturas lógicas de la tabla principal	detalle_factura: 4,504	ivw_ingreso_habitacion: 10	99.8 %
Lecturas lógicas totales	4,597	59	98.7 %
CPU en ejecución	22 ms	2 ms	91 %
Tiempo transcurrido	22 ms	1 ms	95 %

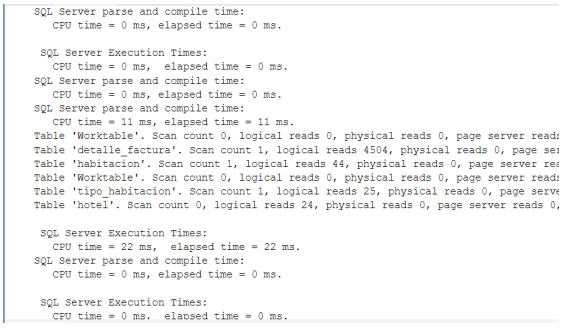


Figura 12a. Consulta 3 sin índice.

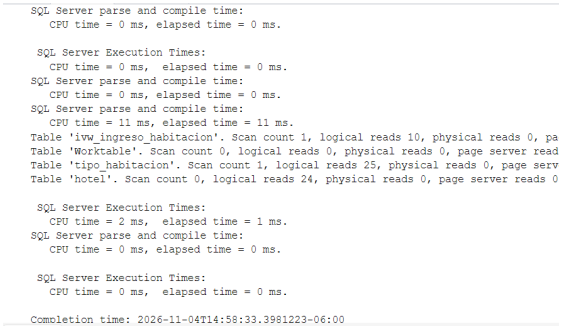


Figura 12b. Consulta 3 con vista indexada.

5.6 Planes de ejecución

Para entender, comparar y explicar cada plan.

Resumen Comparativo

Plan	Costo	Tiempo	CPU	Memoria concedida	Lecturas lógicas (aprox.)
C1 sin índice	**1.502**	**41 ms**	**41 ms**	**4.6 MB**	**1,519**
C1 optimizado	**0.072**	**4 ms**	**4 ms**	**4.6 MB**	**32**
C2 sin índice	**6.021**	**89 ms**	**585 ms**	**187 MB (DOP 12)**	**6,728**
C2 optimizado	**0.474**	**2 ms**	**2 ms**	**6.4 MB (DOP 1)**	**544**

C3 sin índice	**4.411**	**23 ms**	**22 ms**	**137 MB (usó 2.7 MB)**	**4,596**
C3 optimizado	**0.024**	**< 1 ms**	**0 ms**	**2.2 MB**	**59**

Nota: la base es chica (~200 mil facturas), así que los milisegundos salen bajos de por sí. Lo que importa es que las lecturas y la memoria crecen con los datos: con millones de filas, el plan sin índice se pondría mucho peor que el optimizado.

Caso 1: ranking de gasto de huéspedes

Consulta: SELECT * FROM facturacion.vw_ranking_gastos_huespedes ORDER BY ranking_global, id_huesped

Sin índice

Para saber cuánto gastó cada huésped, el sistema revisa las 200,309 facturas una por una, las suma por huésped y luego arma el ranking. Es como contar todos los recibos del hotel cada vez que alguien pide la lista de mejores clientes.

- Un **Index Scan** sobre IX_factura_huesped lee 200,309 filas (1,493 lecturas lógicas).
- **Stream Aggregate** agrupa por id_huesped y deja 741 filas. Es el cuello de botella: 38 ms de los 41 ms totales.
- Un **Merge Join** une el resultado con huesped (1,172 filas, 26 lecturas).
- Después hay 3 pasos de Sort + Window Aggregate que calculan los rankings (uno particionado por tipo_documento).

Optimizado

Se creó un resumen ya calculado del gasto por huésped. Ahora el sistema lee 741 renglones en vez de 200 mil recibos.

- Hay una **vista indexada** (ivw_gasto_huesped, índice CIX_ivw_gasto_huesped). El Clustered Index Scan sobre ella lee 741 filas con 6 lecturas.
- El agregado desapareció del plan. Los Sort y Window Aggregate siguen, pero trabajan sobre pocos datos ya agregados.
- El Merge Join pasó de 39 ms a 1 ms.

> El costo baja ~95 %, las lecturas ~47 veces y el tiempo ~10 veces. El plan sin índice ya usaba un índice no clustered y aun así seguía lento, porque el cuello de botella era calcular la agregación y no encontrar los datos.

Caso 2: variación mensual de ingresos por hotel

Consulta original: SELECT * FROM facturacion.vw_variacion_mensual_ingresos_hotel WHERE mes >= @1 AND mes < @2 ORDER BY hotel, mes (@1 = 2026-03-01, @2 = 2026-06-01)

Sin índice

Se pide la variación de ingresos de marzo a mayo (solo 10 filas). Para responder, el sistema calcula todos los meses de toda la historia de todos los hoteles y recién al final descarta lo que no se pidió.

- **Filtro tardío:** el Filter sobre el mes aparece en el nodo 5, cuando ya se procesaron ~200,340 filas. No puede empujarse hacia abajo porque la vista usa una función de ventana (probablemente LAG, que compara con el mes anterior).

- **Clustered Index Scan** de factura (200,309 filas) y detalle_factura (200,388 filas), con hash joins y hash aggregates.
- **Paralelismo DOP 12:** 585 ms de CPU para 89 ms de reloj.
- **Memoria:** pide 187 MB y usa 84 MB.
- **Waits dominantes:** BPSORT (278 ms) y RESERVED_MEMORY_ALLOCATION (118 ms).
- Estimó 17,429 filas y devolvió 10.

Optimizado

Ahora la consulta va directo a las facturas de febrero a mayo, en vez de recorrerlas todas.

- Se cambió la vista por una función con parámetros de fecha (`fn_variacion_mensual_ingresos_hotel`). Aquí cambió el diseño; un índice solo no alcanzaba.
- Un Index Seek en IX_factura_fecha lee solo 94 facturas, con rango 2026-02-01 a 2026-06-01. Se incluye febrero para poder calcular el LAG de marzo; un Filter posterior descarta febrero.
- IX_detfac_factura permite Nested Loops con seeks hacia detalle_factura, y se usa pk_habitacion para llegar a la habitación.
- Plan serial (DOP 1), 6 MB de memoria y 2 ms de duración.

Caso 3: Top 3 habitaciones más rentables por hotel

Consulta: SELECT * FROM facturacion.vw_top3_habitaciones_rentables ORDER BY hotel, posicion

Sin índice

Para encontrar las 3 mejores habitaciones por hotel, el sistema suma todos los ingresos de las 200 mil líneas de factura y luego se queda con solo 12 filas.

- **Clustered Index Scan** sobre detalle_factura: 201,758 filas leídas. Un Hash Match Aggregate suma por habitación.
- **Sort + Window Aggregate** (tipo ROW_NUMBER) numera las 1,021 habitaciones; un **Filter** "<= 3" deja solo 12.
- **Excessive Grant:** se reservaron 137 MB y se usaron 2.7 MB. Esto puede hacer esperar a otras consultas.
- Hay dos **Adaptive Join**. Eligieron la rama de Nested Loops con 12 seeks; las ramas de scan (hash) se ejecutaron 0 veces.

Optimizado

Hay una tabla resumen de ingresos por habitación que se mantiene sola. Ya no se suman 200 mil líneas cada vez.

- Una **vista indexada** (ivw_ingreso_habitacion, índice IX_ivw_ingreso_habitacion_top) entrega los ingresos ya agregados con 10 lecturas.
- El Window Aggregate y el Filter "<= 3" se mantienen; los joins pasaron a Nested Loops livianos.
- La memoria baja de 137 MB a 2.2 MB y las lecturas de ~4,600 a ~59.

> Ojo: igual se leen las 1,021 filas del índice para devolver 12. Se podría afinar más, pero a esta escala ya no vale la pena.

5.7. Resumen y conclusiones

Consulta	Técnica analítica	Lecturas lógicas (sin → con)	Reducción	CPU (sin → con)
1. Ranking de huéspedes	DENSE_RANK, RANK, SUM OVER	1,964 → 32	98.4 %	21 → 5 ms
2. Variación mensual	LAG	6,794 → 618	90.9 %	620 → 4 ms
3. Top 3 habitaciones	ROW_NUMBER	4,597 → 59	98.7 %	22 → 2 ms

- Con un volumen realista (~200,000 facturas), los índices de cobertura y las vistas indexadas bajan entre 91 % y 99 % las lecturas lógicas de las tres consultas.
- Los milisegundos cambian de una ejecución a otra, por eso comparamos con lecturas lógicas.
- **Lo que cuesta:** cada índice y cada vista indexada ocupa espacio y le da más trabajo a las escrituras (INSERT y UPDATE de facturas). Ese costo lo contamos en el dimensionamiento (sección 6). Y con `SCHEMABINDING` no se puede modificar una tabla base sin eliminar antes la vista.

6. Estrategia de dimensionamiento

6.1. Método de cálculo

Para calcular el tamaño de cada tabla usamos el método que Microsoft documenta para estimar tablas e índices en SQL Server, y lo **ajustamos con las mediciones reales** de los planes de ejecución de la sección 5.6. Una página mide 8,192 B y 8,096 B son útiles (los otros 96 B son el encabezado). Tamaños por tipo: `bigint` 8 B, `int` 4 B, `bit` 1 B, `date` 3 B, `datetime2(3)` 7 B, `decimal(≤9)` 5 B, `decimal(10–19)` 9 B y `nvarchar` 2 B por carácter más 2 B de desplazamiento por columna.

```
Fila (B)      = F + [2 + 2·V + D] + [2 + ⌊(C + 7) / 8⌋] + 4
Filas/página = ⌊ 8096 / (Fila + 2) ⌋
Páginas       = ⌈ Filas ÷ Filas/página ⌉          Tamaño = Páginas × 8 KB
Fila hoja NC = Σ bytes(clave + INCLUDE + clave agrupada) + [2 + ⌊(C + 7) / 8⌋] + 1
Páginas NC    = ⌈ Filas ÷ ⌊8096 / (Fila hoja + 2)⌋ ⌉ × S  +  páginas no hoja
```

F son los bytes de columnas fijas, V las columnas variables, D los bytes de datos variables y C el total de columnas. El 4 es el encabezado de fila y el 2 la entrada en el arreglo de ranuras. S es el factor de densidad de los índices (sección 6.5). Las páginas no hoja se calculan nivel por nivel con ⌈P ÷ ⌊8096/(clave+9)⌋⌉ hasta llegar a una sola página.

6.2. Tamaño de fila por tabla

Tabla	Fijos F (B)	Variables (car. prom.)	Fila = F + var + bitmap + 4	Filas/pág.
catalogo.hotel	id 8 + int 4 + dec 5 = 17	nombre 25, dirección 45, descripción 150	17 + 448 + 3 + 4 = **472**	17
catalogo.tipo_habitacion	id 8 = 8	tipo 18	8 + 40 + 3 + 4 = **55**	142
catalogo.tipo_comodidad	id 8 = 8	tipo 20	8 + 44 + 3 + 4 = **59**	132
catalogo.tipo_empleado	id 8 = 8	tipo 18	8 + 40 + 3 + 4 = **55**	142
catalogo.servicio	id 8 + dec 9 = 17	tipo_servicio 25	17 + 54 + 3 + 4 = **78**	101
catalogo.descuento	id 8 + dec 5 + int 4 = 17	—	17 + 0 + 3 + 4 = **24**	311
catalogo.aumento_costos	id 8 + dec 5 + 2 date 6 + bit 1 = 20	nombre_temporada 25	20 + 54 + 3 + 4 = **81**	97
catalogo.comodidad_tipo_habitacion	3 bigint = 24	detalle 60	24 + 124 + 3 + 4 = **155**	51
operacion.habitacion	3 bigint 24 + 3 int 12 + dec 9 = 45	estado 9, descripción 80	45 + 184 + 4 + 4 = **237**	33
operacion.empleado	2 bigint 16 + dec 9 = 25	nombre 25, correo 30, teléfono 12, dui 10	25 + 164 + 3 + 4 = **196**	40
operacion.huesped	id 8 = 8	nombre 22, correo 28, tel. 12, doc. 10, tipo 6	8 + 168 + 3 + 4 = **183**	43
operacion.reservacion	3 bigint 24 + int 4 = 28	estado 10	28 + 24 + 3 + 4 = **59**	132
operacion.detalle_reservacion	3 bigint 24 + int 4 + 2 date 6 = 34	—	34 + 0 + 3 + 4 = **41**	188
operacion.estadia	2 bigint 16 + 2 dt2 14 = 30	—	30 + 0 + 3 + 4 = **37**	207
operacion.consumo_servicio	4 bigint 32 + dt2 7 = 39	—	39 + 0 + 3 + 4 = **46**	168
facturacion.factura	4 bigint 32 + dt2 7 + dec 9 = 48	metodo_pago 8	48 + 20 + 3 + 4 = **75**	105
facturacion.detalle_factura	6 bigint 48 + int 4 + 5 dec 45 = 97	concepto 30	97 + 64 + 4 + 4 = **169**	47
calidad.resenia	3 bigint 24 + dec 5 = 29	comentario 120	29 + 244 + 3 + 4 = **280**	28

Variables = 2 + 2·V + 2·caracteres. Bitmap de nulos = 2 + ⌊(C+7)/8⌋. Ejemplo con `factura`: 48 + (2 + 2·1 + 2·8) + (2 + ⌊14/8⌋) + 4 = 75 B → ⌊8096 ÷ 77⌋ = 105 filas por página.

6.3. Volumen supuesto y tamaño de datos (año 1, operación media)

Tomamos como referencia un hotel con **≈ 100 facturas al día** (36,500 al año, una por estadía). Las proporciones entre tablas vienen de los datos reales (1,400 líneas de detalle ÷ 190 facturas ≈ 7.4 líneas por factura) y de los conteos de los planes de ejecución (1,172 huéspedes y 1,021 habitaciones). Lo demás son supuestos nuestros para planificar.

Tabla	Supuesto	Filas	Páginas = ⌈Filas ÷ Filas/pág.⌉	× 1.06	Datos (KB)
catalogo.hotel	5 hoteles	5	⌈5 ÷ 17⌉ = 1	2	16
catalogo.tipo_habitacion	8 tipos	8	⌈8 ÷ 142⌉ = 1	2	16
catalogo.tipo_comodidad	12 tipos	12	⌈12 ÷ 132⌉ = 1	2	16
catalogo.tipo_empleado	8 tipos	8	⌈8 ÷ 142⌉ = 1	2	16
catalogo.servicio	20 servicios	20	⌈20 ÷ 101⌉ = 1	2	16
catalogo.descuento	6 reglas	6	⌈6 ÷ 311⌉ = 1	2	16
catalogo.aumento_costos	12 temporadas	12	⌈12 ÷ 97⌉ = 1	2	16
catalogo.comodidad_tipo_habitacion	8 tipos × 5 = 40	40	⌈40 ÷ 51⌉ = 1	2	16
operacion.habitacion	1,021 (plan consulta 3)	1,021	⌈1,021 ÷ 33⌉ = 31	33	264
operacion.empleado	60 empleados	60	⌈60 ÷ 40⌉ = 2	3	24
operacion.huesped	0.70 × 36,500	25,550	⌈25,550 ÷ 43⌉ = 595	631	5,048
operacion.reservacion	1.10 × 36,500	40,150	⌈40,150 ÷ 132⌉ = 305	324	2,592
operacion.detalle_reservacion	1.30 × 40,150	52,195	⌈52,195 ÷ 188⌉ = 278	295	2,360
operacion.estadia	1 × 36,500	36,500	⌈36,500 ÷ 207⌉ = 177	188	1,504
operacion.consumo_servicio	6.1 × 36,500	222,650	⌈222,650 ÷ 168⌉ = 1,326	1,406	11,248
facturacion.factura	100/día × 365	36,500	⌈36,500 ÷ 105⌉ = 348	369	2,952
facturacion.detalle_factura	7.4 × 36,500	270,100	⌈270,100 ÷ 47⌉ = 5,747	6,092	48,736
calidad.resenia	0.30 × 36,500	10,950	⌈10,950 ÷ 28⌉ = 392	416	3,328

Factor de calibración 1.06: multiplicamos por 1.06 las páginas teóricas porque lo medido sale entre 5 y 7 % por encima de la fórmula (sección 6.4).

6.4. Validación de las fórmulas con mediciones reales

Objeto	Filas medidas	Páginas por fórmula	Páginas medidas	Medido ÷ fórmula
factura (datos)	200,309	⌈200,309 ÷ 105⌉ = 1,908	2,038	1.068
detalle_factura (datos)	201,758	⌈201,758 ÷ 47⌉ = 4,293	4,504	1.049
huesped (datos)	1,172	⌈1,172 ÷ 43⌉ = 28	26	0.93
ivw_gasto_huesped	741	⌈741 ÷ 192⌉ + 1 raíz = 5	6	1.20
IX_ivw_ingreso_habitacion_top	1,021	⌈1,021 ÷ 137⌉ + 1 raíz = 9	10	1.11
IX_factura_huesped (NC)	200,309	⌈200,309 ÷ 261⌉ + 3 = 771	1,493	1.94

Las dos tablas grandes crecen en orden (su índice agrupado usa identity) y dan 1.068 y 1.049. El promedio, **1.06**, es nuestro factor de calibración para los datos. `IX_factura_huesped`, en cambio, sale 1.94 veces más grande que la fórmula. Pasa cuando las filas entran con las claves en desorden, y lo explicamos en la sección 6.5.

6.5. Factor por índices: qué es y cómo se calcula

Un índice no agrupado es una copia ordenada de sus columnas clave, de las columnas INCLUDE y de la clave del índice agrupado (8 B que sirven para ubicar la fila). Por eso ocupa espacio extra. Lo medimos así:

```
F_idx = páginas de todos los índices no agrupados de la tabla ÷ páginas de datos ajustadas
Tamaño total de la tabla = páginas de datos × (1 + F_idx) × 8 KB
```

- **Qué índices contamos:** (1) el índice no agrupado que SQL Server crea solo por cada restricción `UNIQUE` (nombre, correo, dui, documento y las claves compuestas) y (2) los cuatro índices de cobertura de la sección 5. Las llaves foráneas no tienen índice propio en el modelo actual, así que su F_idx es 0.
- **Factor de densidad S:** cuando las claves de un índice llegan en desorden (id_huesped, fecha, correo), las hojas se parten y quedan llenas a un ≈ 69 %. Por eso ocupan 1 ÷ 0.69 = **S = 1.44** veces lo teórico (es el valor típico de un B-tree, 1 ÷ ln 2). Justo después de una carga masiva puede llegar a 1.94 (≈ 52 %; así salió `IX_factura_huesped`) y con una reorganización semanal baja a ≈ 1.11 (90 %). Para dimensionar usamos 1.44, que es el valor prudente.
- **Vistas indexadas:** guardan agregados ya calculados. Las sumamos aparte, como el costo de la optimización.

Índice	Filas	Fila hoja = clave + var + bitmap + 1	Filas/pág.	Hojas	× 1.44	No hoja	Total págs.
catalogo (6 índices UQ)	5–20 c/u	≈ 66 B c/u	≈ 119	1 c/u	2 c/u	0	12
habitacion · UQ (id_hotel, nivel, numero)	1,021	24 + 0 + 3 + 1 = **28**	269	4	6	1	7
empleado · UQ correo	60	8 + 64 + 3 + 1 = **76**	103	1	2	0	2
empleado · UQ dui	60	8 + 24 + 3 + 1 = **36**	213	1	2	0	2
huesped · UQ correo	25,550	8 + 60 + 3 + 1 = **72**	109	235	339	4	343
huesped · UQ documento	25,550	8 + 24 + 3 + 1 = **36**	213	120	173	1	174
factura · UQ (id_empleado, id_huesped, id_estadia)	36,500	32 + 0 + 3 + 1 = **36**	213	172	248	1	249
factura · IX_factura_huesped	36,500	25 + 0 + 3 + 1 = **29**	261	140	202	1	203
factura · IX_factura_fecha	36,500	24 + 0 + 3 + 1 = **28**	269	136	196	1	197
detalle_factura · IX_detfac_habitacion	47,450	25 + 0 + 3 + 1 = **29**	261	182	263	1	264
detalle_factura · IX_detfac_factura	47,450	33 + 0 + 3 + 1 = **37**	207	230	332	1	333
resenia · UQ (id_estadia, id_huesped)	10,950	24 + 0 + 3 + 1 = **28**	269	41	60	1	61

Así sale la fila hoja de `IX_factura_huesped`: clave id_huesped 8 + INCLUDE total_a_pagar 9 + clave agrupada id_factura 8 = 25; con bitmap 3 y 1 más son 29 B → ⌊8096 ÷ 31⌋ = 261 filas por página → ⌈36,500 ÷ 261⌉ = 140 hojas × 1.44 = 202, más 1 página no hoja = 203. Los `IX_detfac_*` son filtrados y solo indexan las líneas de habitación (1.3 por factura = 47,450 filas).

6.6. Factor por índices y tamaño total por tabla (año 1)

Tabla	Págs. datos (×1.06)	Págs. índices	F_idx	Total págs.	Total (KB)
catalogo (8 tablas)	16	12	0.75	72 (piso 9 c/u)	576
operacion.habitacion	33	7	0.21	40	320
operacion.empleado	3	4	1.33	9 (piso 9)	72
operacion.huesped	631	517	0.82	1,148	9,184
operacion.reservacion	324	0	0.00	324	2,592
operacion.detalle_reservacion	295	0	0.00	295	2,360
operacion.estadia	188	0	0.00	188	1,504
operacion.consumo_servicio	1,406	0	0.00	1,406	11,248
facturacion.factura	369	649	1.76	1,018	8,144
facturacion.detalle_factura	6,092	597	0.10	6,689	53,512
calidad.resenia	416	61	0.15	477	3,816
Subtotal tablas	**9,773**	**1,847**	**0.19**	**11,666**	**93,328**
vistas indexadas (3 + 1 índice)	—	335	—	335	2,680
TOTAL AÑO 1	**9,773**	**2,182**	**0.22**	**12,001**	**96,008** (≈ 94 MB)

Cómo leer el factor: `factura` tiene F_idx = 1.76 porque es una tabla angosta (75 B por fila) con tres índices (36 + 29 + 28 B por entrada, × 1.44). `detalle_factura` tiene filas de 169 B y solo llega a 0.10. Entre todos, los índices y las vistas le suman **22 %** a los datos: factor global 1 + 0.22 = **1.22**.

6.7. Proyección a 1, 3 y 5 años

Escenario (datos + índices + vistas)	Año 1	Año 3	Año 5	Año 5 con S = 1.94
Operación media (36,500 facturas/año)	94 MB	279 MB	464 MB	488 MB
Operación alta (200,000 facturas/año)	508 MB	1,521 MB	2,534 MB	2,669 MB

Para proyectar repetimos lo de 6.2 a 6.6 multiplicando las filas por el número de años (los catálogos y `habitacion` no crecen). En operación media salen ≈ 94 MB por año (≈ 2.6 KB por factura, contando líneas, consumos e índices). El factor S pesa poco: ≈ +5 % con S = 1.94 y ≈ −3 % con S = 1.11 si se da mantenimiento semanal.

6.8. Dimensionamiento recomendado

Recurso	Configuración	Cálculo y justificación
Archivo de datos (MDF)	Medio: inicial 1 GB, crecimiento 256 MB. Alto: inicial 4 GB, crecimiento 512 MB	Tamaño del año 5 × 1.30 de holgura: medio 464 × 1.3 = 603 MB; alto 2,534 × 1.3 = 3,295 MB. Crecimiento en MB fijos, no en porcentaje
Archivo de log (LDF)	Inicial 256 MB; crecimiento 128 MB	Regla: el doble de la carga masiva más grande. La de 200,000 facturas pesa ≈ 108 MB (6,542 págs. de datos + índices con S = 1.94), así que 2 × 108 = 216 MB → 256 MB
Disco de datos y log	≥ 2 × el MDF del año 5	Deja crecer sin parar el servicio y reconstruir índices (hace falta espacio libre parecido al del índice más grande)
Disco /backup	10 GB, volumen distinto al de datos	Full comprimido = 0.30 × BD (supuesto). Con 4 Full + 7 Diff + 7 días de logs: medio ≈ 0.7 GB y alto ≈ 3.5 GB; con el doble de holgura, ≈ 1.4 y 7.0 GB

Recurso	Configuración	Cálculo y justificación
Memoria	≥ 2 GB (mínimo de SQL Server en Linux)	El conjunto de trabajo del año 5 medio (464 MB) cabe de sobra en la caché
Mantenimiento y monitoreo	Cada semana, reorganizar (5–30 %) o reconstruir (> 30 %) los índices; vigilar con `DBCC SQLPERF(LOGSPACE)` y avisar al llegar al 80 % de uso	Mantiene S cerca de 1.11 en vez de 1.44 y evita autocrecimientos que nadie planeó

Para comparar lo calculado con lo real en cualquier momento sirve esta consulta (la suma de `reservado_mb` por tabla debería coincidir con las columnas «Total» de 6.6 para el volumen que haya en ese momento):

```
SELECT s.name + '.' + t.name AS tabla,
       SUM(ps.reserved_page_count) * 8 / 1024.0 AS reservado_mb,
       SUM(CASE WHEN ps.index_id IN (0,1) THEN ps.row_count ELSE 0 END) AS filas
FROM sys.dm_db_partition_stats ps
JOIN sys.tables t ON t.object_id = ps.object_id JOIN sys.schemas s ON s.schema_id = t.schema_id
GROUP BY s.name, t.name ORDER BY reservado_mb DESC;
```

7. Estrategia de respaldo y recuperación

7.1. Objetivos de recuperación (RPO y RTO)

Objetivo	Valor	Justificación	Cómo se cumple
RPO (Recovery Point Objective)	15 minutos	La base maneja en tiempo real cosas críticas: facturas, cobros con tarjeta, confirmaciones de reserva y registro de huéspedes. Perder más de 15 minutos significaría perder pagos y vender dos veces la misma habitación.	Modelo de recuperación FULL y respaldo del log de transacciones cada 15 minutos
RTO (Recovery Time Objective)	1 hora	Si el servidor se cae, el hardware falla o se corrompen el MDF o el LDF, recepción puede trabajar un rato con contingencias manuales. Una hora le alcanza al DBA para preparar el entorno y restaurar la cadena de respaldos.	Respaldos comprimidos y procedimiento de restauración encadenada Full + Diff + Logs documentado y probado

Relación entre RPO, RTO, WRT y MTD

Junto al RPO y al RTO hay otras dos métricas que ayudan a entender la continuidad del servicio: el WRT (Work Recovery Time) y el MTD (Maximum Tolerable Downtime). El esquema siguiente las pone en línea de tiempo desde la última copia de seguridad. El RPO dice cuántos datos se pierden, el RTO cuánto tarda en volver la información, el WRT cuánto tarda el servicio en normalizarse y el MTD cuánto tiempo puede estar caído el sistema antes de que las consecuencias sean inaceptables.

¿Cómo se relacionan RPO, RTO, WRT y MTD?



Esquema 7.1. Relación temporal entre RPO, RTO, WRT y MTD a partir de una caída del sistema.

Métrica	Qué describe	Aplicación en CÉNIT
RPO (Recovery Point Objective)	Cuántos datos se pueden perder: desde la última copia de seguridad hasta la caída del sistema.	15 minutos: modelo de recuperación FULL y respaldo del log cada 15 minutos.
RTO (Recovery Time Objective)	Cuánto tiempo pasa hasta que los datos están restaurados después de la caída.	≈ 25–30 min estimados: diagnóstico (15), Full y Diferencial (5) y logs posteriores (5–10).
WRT (Work Recovery Time)	Tiempo en el que se recuperará el servicio real mediante la verificación de los sistemas y la puesta en línea, tras haberlos recuperado en la parte del RTO.	≈ 15 min estimados: DBCC CHECKDB y validación de conteos (5) y reconexión de aplicaciones y pruebas funcionales (10).
MTD (Maximum Tolerable Downtime)	Tiempo máximo que puede estar inoperativo el servicio antes de que las consecuencias sean inaceptables para la empresa. Debe cumplirse RTO + WRT ≤ MTD.	1 hora (límite de la sección 7.1). El total estimado de 40–45 min queda dentro de ese límite.

Nota: la 1 hora de la tabla anterior es el tope para volver a operar por completo (RTO + WRT), es decir, el MTD. Los tiempos parciales de la sección 7.4 se reparten entre el RTO (restaurar los datos) y el WRT (verificar y poner en línea).

Estos valores cambian según el servicio. No pesa igual un proceso transaccional de producción (reservaciones y facturación) que uno administrativo o analítico (reportes y dashboard), así que sus objetivos de recuperación pueden ser distintos. Conviene clasificar los servicios y ver qué impacto tendría su caída en el negocio.

7.2. Esquema de respaldos automatizados

La base trabaja en modelo de recuperación FULL (`ALTER DATABASE HotelDB SET RECOVERY FULL`). Los respaldos corren solos con tres jobs de SQL Server Agent (en `msdb`). El archivo de cada respaldo lleva fecha y hora en el nombre (`yyyymmdd_hhmmss`), así nunca se pisa una copia anterior.

Tipo	Job	Frecuencia	Archivo / opciones	Retención recomendada
Completo (Full)	HotelDB_Auto_Backup_Full	Semanal: lunes 00:00	HotelDB_Full_<fecha>.bak · FORMAT, COMPRESSION	4 semanas
Diferencial	HotelDB_Auto_Backup_Diff	Diario 01:00	HotelDB_Diff_<fecha>.bak · FORMAT, COMPRESSION, DIFFERENTIAL	7 días
Log de transacciones	HotelDB_Auto_Backup_Log	Cada 15 minutos, todo el día	HotelDB_Log_<fecha>.trn · COMPRESSION, NOINIT	7 días

```
EXEC dbo.sp_add_job @job_name = N'HotelDB_Auto_Backup_Log', @enabled = 1;
EXEC dbo.sp_add_jobstep @job_name = N'HotelDB_Auto_Backup_Log', @step_name = N'Paso_Backup_Log',
  @subsystem = N'TSQL', @database_name = N'master', @command = N'
  DECLARE @Archivo NVARCHAR(500) = N''/backup/HotelDB_Log_''
    + CONVERT(VARCHAR(8), GETDATE(), 112) + ''_'' + REPLACE(CONVERT(VARCHAR(8), GETDATE(), 108),
''':'', '''') + ''.trn'';
  BACKUP LOG [HotelDB] TO DISK = @Archivo WITH COMPRESSION, NOINIT, STATS = 10;';
EXEC dbo.sp_add_jobschedule @job_name = N'HotelDB_Auto_Backup_Log', @name = N'Cada_15_Minutos',
  @freq_type = 4, @freq_interval = 1, @freq_subday_type = 4, @freq_subday_interval = 15,
  @active_start_time = 000000;
EXEC dbo.sp_add_jobserver @job_name = N'HotelDB_Auto_Backup_Log';
```

Los jobs Full y Diferencial tienen la misma estructura (Full: `freq_type = 8`, `freq_interval = 2`; Diferencial: `freq_type = 4`, a la 01:00). Cada job se probó con `sp_start_job`.

7.3. Procedimiento de recuperación

Recuperar es restaurar en cadena: el último Full, el último Diferencial y todos los logs que vinieron después, en orden cronológico. Solo el último paso lleva `WITH RECOVERY`. Con `STOPAT` se puede volver a un instante exacto, por ejemplo justo antes de un borrado por error.

```
USE master;
ALTER DATABASE HotelDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;   -- desconectar sesiones

RESTORE DATABASE HotelDB FROM DISK = '/backup/HotelDB_Full_<fecha>.bak'
  WITH NORECOVERY, REPLACE, STATS = 10;
RESTORE DATABASE HotelDB FROM DISK = '/backup/HotelDB_Diff_<fecha>.bak'
  WITH NORECOVERY, STATS = 10;
RESTORE LOG HotelDB FROM DISK = '/backup/HotelDB_Log_<fecha1>.trn' WITH NORECOVERY;
RESTORE LOG HotelDB FROM DISK = '/backup/HotelDB_Log_<fechaN>.trn' WITH RECOVERY;  -- último log
                                                                   -- (STOPAT = '<fecha hora>'
opcional)
ALTER DATABASE HotelDB SET MULTI_USER;

-- Verificación posterior
USE HotelDB;
SELECT 'Huéspedes' AS Tabla, COUNT(*) AS Total FROM operacion.huesped
UNION ALL SELECT 'Reservaciones', COUNT(*) FROM operacion.reservacion
UNION ALL SELECT 'Facturas', COUNT(*) FROM facturacion.factura;
DBCC CHECKDB ('HotelDB');
```

Prueba que hicimos: el script de validación inserta un huésped nuevo antes de los respaldos manuales (Full, Diferencial y Log), restaura toda la cadena y revisa los conteos de huéspedes, reservaciones y facturas. Al final corre `DBCC CHECKDB` para confirmar que la base quedó íntegra.

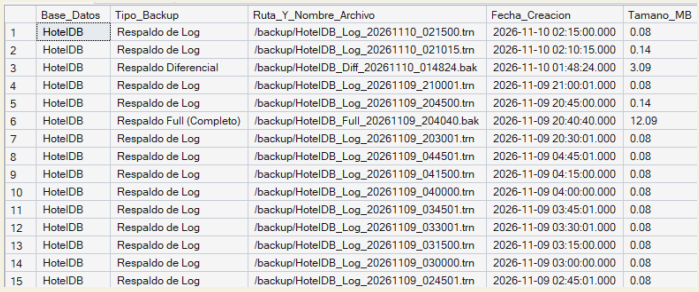
7.4. Cumplimiento del RTO

Paso	Tiempo estimado
Diagnóstico y aprovisionamiento del servidor o instancia	15 min
Restauración del Full y del Diferencial (base de pocos cientos de MB, respaldos comprimidos)	5 min
Aplicación de los logs posteriores (hasta 96 por día)	5–10 min
`DBCC CHECKDB` y validación de conteos	5 min
Reconexión de aplicaciones y pruebas funcionales	10 min
Total estimado	**≈ 40–45 min (menor a 60 min)**



	Nombre_Job	Paso	Fecha_Hora_Ejecucion	Estado_Ejecucion	Duracion
1	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-10 02:10:15.000	Exitoso	00:00:01
2	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-10 02:10:15.000	Exitoso	00:00:01
3	HotelDB_Auto_Backup_Diff	Paso_Backup_Diff	2026-11-10 01:48:24.000	Exitoso	00:00:00
4	HotelDB_Auto_Backup_Diff	(Job outcome)	2026-11-10 01:48:24.000	Exitoso	00:00:00
5	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 21:00:00.000	Exitoso	00:00:01
6	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-09 21:00:00.000	Exitoso	00:00:01
7	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 20:45:00.000	Exitoso	00:00:00
8	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-09 20:45:00.000	Exitoso	00:00:00
9	HotelDB_Auto_Backup_Full	Paso_Backup_Full	2026-11-09 20:40:40.000	Exitoso	00:00:00
10	HotelDB_Auto_Backup_Full	(Job outcome)	2026-11-09 20:40:40.000	Exitoso	00:00:00
11	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 20:30:01.000	Exitoso	00:00:00
12	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-09 20:30:01.000	Exitoso	00:00:00
13	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 04:45:01.000	Exitoso	00:00:00
14	HotelDB_Auto_Backup_Log	(Job outcome)	2026-11-09 04:45:01.000	Exitoso	00:00:00
15	HotelDB_Auto_Backup_Log	Paso_Backup_Log	2026-11-09 04:30:01.000	Fallido	00:00:00

Figura 13. Jobs de respaldo en SQL Server Agent (HotelDB_Auto_Backup_Full / Diff / Log) y su historial de ejecución.



	Base_Datos	Tipo_Backup	Ruta_Y_Nombre_Archivo	Fecha_Creacion	Tamano_MB
1	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261110_021500.trn	2026-11-10 02:15:00.000	0.08
2	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261110_021015.trn	2026-11-10 02:10:15.000	0.14
3	HotelDB	Respaldo Diferencial	/backup/HotelDB_Diff_20261110_014824.bak	2026-11-10 01:48:24.000	3.09
4	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_210001.trn	2026-11-09 21:00:01.000	0.08
5	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_204500.trn	2026-11-09 20:45:00.000	0.14
6	HotelDB	Respaldo Full (Completo)	/backup/HotelDB_Full_20261109_204040.bak	2026-11-09 20:40:40.000	12.09
7	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_203001.trn	2026-11-09 20:30:01.000	0.08
8	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_044501.trn	2026-11-09 04:45:01.000	0.08
9	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_041500.trn	2026-11-09 04:15:00.000	0.08
10	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_040000.trn	2026-11-09 04:00:00.000	0.08
11	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_034501.trn	2026-11-09 03:45:01.000	0.08
12	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_033001.trn	2026-11-09 03:30:01.000	0.08
13	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_031500.trn	2026-11-09 03:15:00.000	0.08
14	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_030000.trn	2026-11-09 03:00:00.000	0.08
15	HotelDB	Respaldo de Log	/backup/HotelDB_Log_20261109_024501.trn	2026-11-09 02:45:01.000	0.08

Figura 14. Historial de backups Generrados

8. Dashboard en Power BI

8.1. Objetivo y fuente de datos

El dashboard le da a la gerencia una vista completa de cómo va el hotel: ingresos, ocupación, comportamiento de los huéspedes y qué habitaciones dejan más. Power BI Desktop se conecta a SQL Server (`HotelDB`) y lee las **vistas del esquema** `reportes` y las tres vistas analíticas optimizadas de la sección 5, no las tablas transaccionales. Así respetamos el modelo de seguridad (solo lectura sobre la capa analítica) y aprovechamos los índices que ya creamos.

8.2. Indicadores y visualizaciones

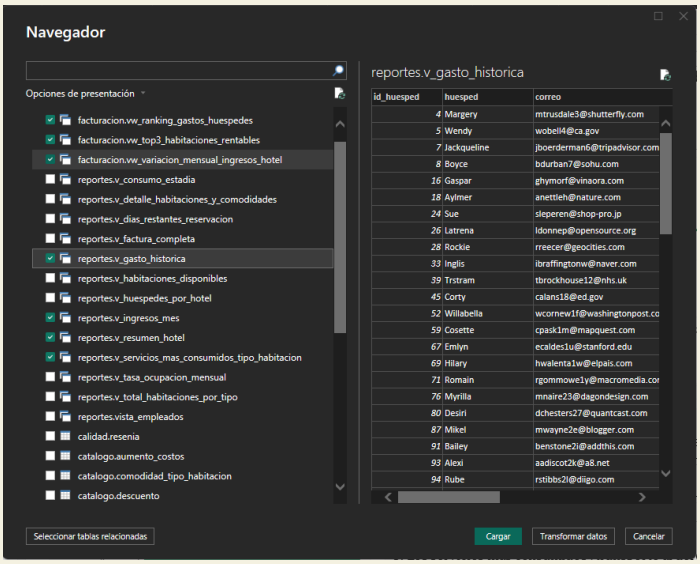
Vista de origen	Indicador	Visualización
reportes.v_ingresos_mes	Ingresos mensuales	Gráfico de columnas / líneas por mes
reportes.v_tasa_ocupacion_mensual	Tasa de ocupación mensual	Gráfico de líneas con porcentaje
reportes.v_resumen_hotel	Resumen por hotel (habitaciones, calificación)	Tarjetas y tabla por hotel
reportes.v_huespedes_por_hotel	Huéspedes por hotel	Gráfico de barras

Vista de origen	Indicador	Visualización
reportes.v_servicios_mas_consumidos_tipo_habitacion	Servicios más consumidos por tipo de habitación	Barras apiladas / matriz
facturacion.vw_ranking_gastos_huespedes	Top de huéspedes por gasto	Tabla ordenada con ranking
facturacion.vw_variacion_mensual_ingresos_hotel	Variación mensual de ingresos (%)	Líneas por hotel con segmentación de fechas
facturacion.vw_top3_habitaciones_rentables	Top 3 habitaciones por hotel	Tabla / matriz por hotel

8.3. Conexión, seguridad y actualización

- La conexión usa un login de solo lectura ligado a un rol con SELECT sobre `reportes` (por ejemplo, el del gerente o el del auditor). Las credenciales no van guardadas dentro del archivo que se comparte.
- **Modo importación:** el volumen es chico y las consultas ya están optimizadas. La actualización programada va en horas de poca ocupación, después del respaldo diferencial de la 01:00.
- Hay segmentaciones por hotel y por rango de fechas que filtran todas las páginas del informe.

8.4. Evidencia del dashboard



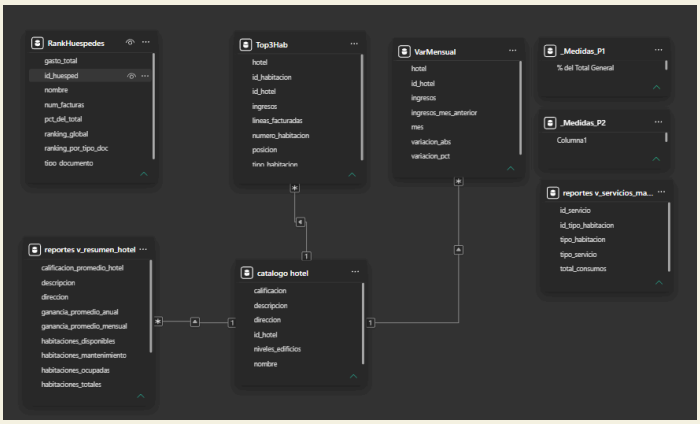


Figura 16. Modelo de datos en Power BI (tablas/vistas importadas y relaciones).



Figura 17. Dashboard – página 1: ingresos, ocupación y variación mensual.

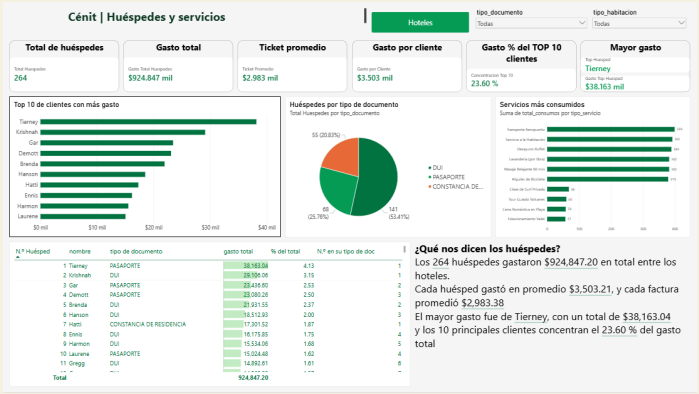


Figura 18. Dashboard – página 2: huéspedes, servicios y habitaciones más rentables.

9. Conclusiones

- El modelo en 3FN, ordenado en esquemas, nos permitió dar permisos por esquema y cuidar las reglas de negocio con restricciones y triggers.
- En seguridad combinamos roles de menor privilegio, un DENY explícito para auditoría y SQL Server Audit sobre facturación, y lo comprobamos con pruebas por rol.
- Con ~200,000 facturas, los índices de cobertura y las vistas indexadas bajaron entre 91 % y 99 % las lecturas lógicas de las consultas analíticas.
- Calculamos el tamaño tabla por tabla: ≈ 94 MB el primer año y ≈ 464 MB al quinto en operación media (≈ 2.5 GB en operación alta), con un factor global por índices de 1.22. Los respaldos Full, Diferencial y Log automáticos sostienen un RPO de 15 minutos y un RTO de 1 hora.
- El dashboard en Power BI se alimenta de la capa analítica y no toca las tablas transaccionales.

Anexo A. Calendario de políticas de backup – HotelDB

Calendario de planificación que armó el equipo; complementa la sección 7.

Elemento	Valor
Base de datos y modelo	HotelDB · modelo de recuperación FULL
Objetivos de recuperación	RPO ≤ 15 min · RTO < 30 min (meta operativa del calendario)
Automatización	3 jobs activos de SQL Server Agent (Full / Diff / Log)
Volumen semanal estimado	680 backups por semana (1 Full, 7 Diff, 672 Log)

Nota: en la sección 7.1 el RTO es de 1 hora como límite aceptable. El calendario trabaja con una meta interna más exigente (< 30 min).

A.1. Políticas de backup y automatización de jobs

Tipo	Job SQL Agent	Frecuencia / hora	Ruta y patrón de archivo	Parámetros	Propósito y RPO
Full (completo)	HotelDB_Auto_Backup_Full	Semanal: lunes 00:00	/backup/HotelDB_Full_YYYYMMDD_HHMMSS.bak	FORMAT, COMPRESSION	Línea base integral del sistema
Diferencial	HotelDB_Auto_Backup_Diff	Diario (lun–dom) 01:00	/backup/HotelDB_Diff_YYYYMMDD_HHMMSS.bak	FORMAT, COMPRESSION, DIFFERENTIAL	Cambios acumulados desde el último Full
Log de transacciones	HotelDB_Auto_Backup_Log	Continuo cada 15 min, 24/7 (00:00 a 23:45)	/backup/HotelDB_Log_YYYYMMDD_HHMMSS.trn	COMPRESSION, NOINIT	RPO ≤ 15 min; trunca el log inactivo

A.2. Calendario semanal

Franja horaria	Lunes	Martes a domingo
00:00 – 00:30 (ventana crítica)	FULL BACKUP (00:00) + Log (00:00, 00:15)	Log Backup (00:00, 00:15)
00:30 – 01:00 (madrugada temprana)	Log Backup (00:30, 00:45)	Log Backup (00:30, 00:45)
01:00 – 01:30 (ventana diferencial)	DIFERENCIAL (01:00) + Log	DIFERENCIAL (01:00) + Log
01:30 – 06:00 (madrugada valle)	Log Backup cada 15 min (18 ejecuciones)	Log Backup cada 15 min (18 ejecuciones)
06:00 – 12:00 (turno mañana / check-out)	Log Backup cada 15 min (24 ejecuciones)	Log Backup cada 15 min (24 ejecuciones)
12:00 – 18:00 (turno tarde / check-in)	Log Backup cada 15 min (24 ejecuciones)	Log Backup cada 15 min (24 ejecuciones)
18:00 – 23:45 (turno noche / cierre)	Log Backup cada 15 min (24 ejecuciones)	Log Backup cada 15 min (24 ejecuciones)
Total diario (semana: 98 + 6 × 97 = 680)	**98** (1 Full, 1 Diff, 96 Log)	**97** por día (1 Diff, 96 Log)

A.3. Guía de restauración de emergencia

Paso	Archivo requerido	Cláusula	Instrucción T-SQL	Estado de HotelDB
1 · Full	HotelDB_Full_YYYYMMDD_HHMMSS.bak	NORECOVERY, REPLACE	`RESTORE DATABASE HotelDB FROM DISK = '/backup/HotelDB_Full_20250217_000000.bak' WITH NORECOVERY, REPLACE;`	RESTORING (inaccesible)
2 · Diferencial	HotelDB_Diff_YYYYMMDD_HHMMSS.bak	NORECOVERY	`RESTORE DATABASE HotelDB FROM DISK = '/backup/HotelDB_Diff_20250219_010000.bak' WITH NORECOVERY;`	RESTORING (avanza LSN)
3 · Logs intermedios	HotelDB_Log_YYYYMMDD_HHMMSS.trn	NORECOVERY	`RESTORE LOG HotelDB FROM DISK = '/backup/HotelDB_Log_20250219_011500.trn' WITH NORECOVERY;` (repetir en orden cronológico)	RESTORING (aplica logs)
4 · Último log	HotelDB_Log_YYYYMMDD_HHMMSS.trn (cola final)	RECOVERY	`RESTORE LOG HotelDB FROM DISK = '/backup/HotelDB_Log_20250219_143000.trn' WITH RECOVERY;`	ONLINE (lista para producción)

Regla de oro de la recuperación ante desastres

- La cadena de recuperación (LSN) no se puede romper: primero el backup Full más reciente, luego solo el último Diferencial y después todos los logs generados desde ese diferencial, en estricto orden cronológico.
- Todos los pasos anteriores llevan `WITH NORECOVERY`. Solo el último comando de log usa `WITH RECOVERY`, que es el que deja la base ONLINE.